

Practical extraction and reporting language (Perl)

Nathan Warner



Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1	Perl	3
2	Comments	5
3	Command line options	6
4	Variables	7
5	Arrays	13
6	Hashes	16
7	Special variables	18
8	More on strings	21
9	Operators and functions	22
9.1	Quote operators and siblings	24
9.2	Important builtin functions	25
10	Constructs	27
11	Intro to Perl IO	34
11.1	File IO	36
11.2	Directory Interaction	44
12	Subroutines	49
12.1	Overloading	59
13	Regular expressions	61
13.1	Advanced Regex	68

14	References and complex data structures	71
15	External Files and Packages	76
16	Modules	80
17	Using Perl Objects (Perl OOP)	89
18	Threads	95
19	System interaction	101
20	Command line switches	106
21	Idioms	108
21.1	Secret Operators	109
22	Standard functions	114
23	Perl examples	116

Perl

- **Practical extraction and reporting language (perl):** Perl is a high-level, general-purpose, interpreted, dynamic programming language. Though Perl is not officially an acronym, **Practical Extraction and Reporting Language** is commonly used as a **backronym**.
- **use version pragma:** The use VERSION pragma in Perl enforces a minimum version required to run a script and automatically enables modern language features

You should place this declaration at the very top of your file. Modern Perl code uses the vX.Y v-string format rather than old-style decimals:

```
0 use v5.36
```

Versions v5.10 and higher automatically activate the feature pragma for that release. For example, use v5.36; is equivalent to loading use feature ':5.36';. This gives you immediate access to built-in keywords like say, state, and updated signatures without typing extra code

Declaring use v5.11 or higher automatically turns on the strict pragma (use strict;) lexically across your file.

Declaring use v5.35 or higher automatically turns on the warnings pragma (use warnings;

- **use feature:** In Perl, the use feature pragma allows you to enable newer syntactic features and capabilities without breaking backward compatibility for older programs. Because it is a lexical pragma, any feature you enable is only active from the line it is declared until the end of its enclosing code block.
 - **Individual Features:** Pass a list of feature names to the pragma.

```
0 use feature 'say';  
1 say "Hello, world!"; # Appends a newline
```

- **Feature bundles:** Load all features made available in a specific Perl release by prefixing the version with a colon

```
0 use feature ':5.36'
```

- **Implicit loading:** Requiring a minimum Perl version automatically enables the feature bundle for that version
- **Command Line Execution:** Use the -E flag instead of -e to quickly test code snippets with all features enabled for your current Perl version.

```
0 perl -E 'say "Fast one-liner testing!'"
```

- **Using the diagnostics pragma:** Another pragma that you may find useful is the diagnostics pragma. This translates the normally terse diagnostics emitted from the perl compiler and the perl interpreter into much more useful ones

```
o use diagnostics;
```

The diagnostics pragma makes your warnings much more verbose, and it slows the start-up time of your script considerably. You should remove it before putting your code into production.

- **Memory management:** Perl 5 memory management is primarily based on reference counting, with a runtime that manages Perl values such as scalars, arrays, hashes, and objects for you.
- **Compiled?:** Every time you execute a script, the Perl interpreter first compiles your source code into an internal bytecode before immediately running it. Because this compilation happens seamlessly in the background right before execution, Perl is often colloquially categorized as an interpreted scripting language.
 - **Compilation Phase:** The engine parses your source code. It converts text into optimized opcode structures.
 - **Execution Phase:** The internal Perl virtual machine runs that bytecode.
- **Pass by reference:** Arguments in Perl are passed by reference implicitly under the hood. Unlike many modern languages that default to copying a value, Perl passes the actual underlying variable into the function using a special array called `@_`.

When you call `chomp($text)`, Perl doesn't copy the string "Hello\n". Instead, it places an alias of the `$text` variable into the first slot of the argument array (`$_[0]`).

Because `chomp` is an internal C function built into the Perl core, it modifies `$_[0]` directly. Since `$_[0]` points to the exact same memory location as your `$text` variable, your original variable changes.

Comments

- **Standard comments:** Uses a hashtag

```
0  # Comment
```

- **Block comments:** We can also use Plain Old Documentation (POD)

```
0  =begin comment
1  ...
2  ...
3  =end comment
4
5  =cut
```

=cut resumes Perl code execution. "Stop ignoring this text; switch back to executable Perl code."

=end comment closes a =begin comment region. "This specific comment block is done, but continue treating subsequent text as documentation."

- **__END__**: You can signal to Perl the end of your program by using the special **__END__** tag. Anything below **__END__** will be ignored by Perl.

```
0  #!/usr/bin/perl
1
2  print "Hello world";
3
4  __END__
```

Command line options

- **Warnings:** Turn on warnings with -w

```
o  #!/usr/bin/perl -w
```

Variables

- **Pragmatic information (pragma):** A code pragma (short for "pragmatic information") is a special instruction or command embedded in source code. It tells a compiler, interpreter, or assembler how to process the program, but it does not change the actual logic or output of the software itself.
- **Note about variables:** In Perl, it is not necessary to declare your variables before you begin. You can summon a variable into existence simply by using it, and it will be globally available to any routine in your program. If you're used to programming in C or any of a number of other languages, this may seem odd and even dangerous to you. This is indeed the case. That's why you want to use the strict pragma.
- **Variable sigils (prefixes):** In Perl, variable prefixes are called sigils, and they immediately identify the data type of the variable.
 - \$ used for scalars
 - @ used for arrays
 - % used for hashes. A hash is an unordered collection of key-value pairs (associative arrays).
- **my, our, and state:** In Perl, my declares a truly private lexical variable, state creates a persistent static variable that initializes only once, and our creates a temporary shortcut to a global package variable

A lexical variable is a variable that is available only within a specific, textually defined part of a program, such as a function, a block, or a file.

- **The strict pragma:** Once we use the strict pragma, we have to explicitly declare new variables with scope keywords my, our, or state.

```
0  #!/usr/bin/perl -w
1  use strict;
2
3  my $x = 50; # Okay
4  $y = 100; # Bad
```

- **Variables with name a or b:** \$a and \$b are special variables in Perl. they are implicitly used by Perl's built-in sort function. Inside the sort block, Perl automatically places the two values being compared into \$a and \$b. Consequently, strict 'vars' specifically permits \$a and \$b without requiring my
- **Scalars:** A scalar is a single item of data

```
0  my $name = "Arthur";
1  my $whoami = 'Just Another Perl Hacker';
2  my $meaning_of_life = 42;
3  my $number_less_than_1 = 0.000001;
4  my $very_large_number = 3.27e17; # 3.27 by 10 to the power of
   ↪ 17
```

A scalar can be text of any length, and numbers of any precision (machine dependent, of course). Perl doesn't need us to tell it what type of data we're going to put into the scalar. In fact, Perl doesn't care if the type of data in the scalar changes throughout your program. Perl magically converts between them when it needs to

- **Single vs double quotes:** In Perl, quotes are required to distinguish strings from the language's reserved words or other expressions. Either type of quote can be used, but there is one important difference: double quotes can include other variable names inside them, and those variables will then be interpolated

```
0 my $x = 50;  
1 print "$x";
```

- **Special characters:** Special characters, such as the `\n` newline character, are only available within double quotes. Single quotes will fail to expand these special characters just as they fail to expand variable names
- **String concatenation:** Uses the dot notation

```
0 print "Hello, " . "World!";
```

- **Bash style interpolation:**

```
0 my $x = "Hello";  
1 print "${x}, World";
```

- **References / dereferencing:**

```
0 my $x = 10;  
1 my $y = \$x; # Creates a reference  
2  
3 $$y = 25; # $$ Dereferences
```

- `\$scalar`
- `\@array`
- `\%hash`
- `&function`

- **Typeglobs:** In Perl, a typeglob (represented by the asterisk `*` sigil) is an internal data type that holds a complete symbol table entry for a given identifier.

Because Perl allows you to use the exact same name for different variable types simultaneously (e.g., `$apple`, `@apple`, `%apple`, and `sub apple`), Perl uses a typeglob to bundle all of these variables into a single slot in the package's symbol table. You can visualize a typeglob as a structure containing separate internal pointers for each variable type

Slot / Prefix	Data Type	Example
\$	Scalar	\$apple
@	Array	@apple
%	Hash	%apple
&	Subroutine / Code	apple()
*	IO / Filehandle	APPLE

- **Typglob uses:** Assigning one typglob to another maps all variables of that name to the new name.

```

0  *foo = *bar;
1  # Now $foo is an alias for $bar, @foo for @bar, %foo for
   ↪ %bar, etc.

```

By assigning a reference to a typglob slot, you can alias just one specific type

```

0  *foo = \$bar;
1  # Only $foo and $bar are now the same variable.
2  # @foo and @bar remain completely separate.

```

- **Creating constants**

```

0  my $PI = \3.14;

```

creates a reference to a read-only scalar constant containing the value 3.14.

```

0  print $$PI;
1  $$PI = 10; # Error

```

- **Creating constants with typglob:** You can generate read-only variables by assigning a reference to a constant value inside a typglob.

```

0  *PI = \3.14159;
1  $PI = 4; # Compiles to a run-time error: "Modification of a
   ↪ read-only value..."

```

- **Legacy filehandles:** In older Perl code, typglobs or references to typglobs (*FH) were required to pass filehandles into subroutines. Modern Perl handles this automatically via lexical scalar filehandles (open my \$fh, ...).
- **Slots in a typglob:** Use

```
0  *NAME{SLOT}
```

Common slot names include:

```
0  SCALAR
1  ARRAY
2  HASH
3  CODE
4  IO
5  FORMAT
6  GLOB
```

- **Inspecting prototypes:** If you need to retrieve or inspect a prototype programmatically from an existing function, you can use the built-in prototype function:

```
0  my $proto = prototype('my_subroutine_name');
```

- **Variable attributes:** In Perl, a variable attribute is extra metadata attached to a variable declaration. It can change how the variable is created, stored, or handled. Attributes appear after the declared variable:

```
0  my $scalar :Attribute;
1  my @array  :Attribute;
2  my %hash   :Attribute;
```

An attribute can optionally receive an argument:

```
0  my $value :Attribute(argument);
```

Multiple attributes can be attached to one variable:

```
0  my $value :First :Second(argument);
```

Whether an attribute is valid and what it does depends on the module or package that implements it.

- **MODIFY_*__ATTRIBUTES hooks:** This is Perl's fundamental, low-level mechanism. A package defines a special method corresponding to the variable's underlying reference type:

```
0  MODIFY_SCALAR_ATTRIBUTES
1  MODIFY_ARRAY_ATTRIBUTES
2  MODIFY_HASH_ATTRIBUTES
```

```

0  sub MODIFY_SCALAR_ATTRIBUTES {
1      my ($package, $variable_ref, @attributes) = @_;
2
3      my @unrecognized;
4
5      for my $attribute (@attributes) {
6          if ($attribute eq 'Positive') {
7              print "Applying :Positive\n";
8          }
9          else {
10             push @unrecognized, $attribute;
11          }
12      }
13
14      return @unrecognized;
15  }
16
17  my $number :Positive = 10;

```

When Perl compiles the declaration, it approximately performs:

```

0  Example->MODIFY_SCALAR_ATTRIBUTES(
1      \$number,
2      'Positive'
3  );

```

The handler must return every attribute it did not recognize. If it returns an attribute, Perl reports an error

A handler that accepts everything can return an empty list

- **Fetch hooks:** A package may also define a corresponding fetch hook:

```

0  FETCH_SCALAR_ATTRIBUTES
1  FETCH_ARRAY_ATTRIBUTES
2  FETCH_HASH_ATTRIBUTES

```

```

0  sub FETCH_SCALAR_ATTRIBUTES {
1      my ($package, $variable_ref) = @_;
2
3      return 'Positive';
4  }

```

The attributes module can then retrieve them:

```

0  use attributes;
1
2  my @attrs = attributes::get(\$number);

```

- **Attribute::Handlers:** is a higher-level interface built around Perl's attribute machinery. It lets you define a handler using attribute syntax instead of manually implementing each `MODIFY_*_ATTRIBUTES` method.

```

0  package Example;
1
2  use strict;
3  use warnings;
4  use Attribute::Handlers;
5
6  sub Positive :ATTR(SCALAR) {
7      my (
8          $package,
9          $symbol,
10         $variable_ref,
11         $attribute,
12         $data,
13         $phase,
14         $filename,
15         $line
16     ) = @_;
17
18     print "Applied :Positive at $filename line $line\n";
19 }
20
21 my $number :Positive = 10;

```

You can allow several target types

```

0  sub Tracked :ATTR(SCALAR, ARRAY, HASH) {
1      ...
2  }

```

Arrays

- **Creating arrays:** Arrays are initialised by creating a comma separated list of values:

```
0  my @arr = (1,2,3);
```

Perl's arrays, like those in many other languages, are indexed from zero

- **Accessing:** As you would expect

```
0  print $fruits[0];
```

We use the dollar sign here because the indexing produces a scalar.

- **Slicing:** If we wanted to only deal with a portion of the array, we use what we call an array slice. These are written as follows:

```
0  @arr # entire array
1  @arr[1,3]
2  @arr[0..2]
```

- **Negative indexing:**

```
0  print $arr[-1] # 3
```

- **Array size:**

```
0  print $#arr + 1;
```

Note that \$#arr gets the index of the last element. We can also use

```
0  my $size = @arr;
1  print my $size = @arr;
```

- **qw// (quote words):** It takes whitespace separated words and turns them into a list

```
0  my @arr = qw/hello , world/
```

Your list can stretch over multiple lines, and your delimiter doesn't need to be a slash. Whatever punctuation character that you place after the qw becomes the delimiter. So if you prefer parentheses over slashes, that's no problem at all

- **Printing out arrays**

```

0 my @arr = (1,2,3);
1 print @arr;
2 print join(", ", @arr);
3 print "@arr"

```

The first method takes advantage of the fact that print takes a list of arguments to print and prints them out sequentially. The second uses join() which joins an array or list together by separating each element with the first argument. The third option uses double quote interpolation and a little bit of Perl magic to pick which character(s) to separate the words with.

- **Context:** There's a term you've heard used just recently but which hasn't been explained: context. Context refers to how an expression or variable is evaluated in Perl. The two main contexts are:
 - Scalar context
 - List context

Scalar variables are always evaluated in scalar context, however arrays and hashes can be evaluated in both scalar contexts (when we treat them as scalars) and list contexts (when we treat them as arrays and hashes). Scalar variables are always evaluated in scalar context, however arrays and hashes can be evaluated in both scalar contexts (when we treat them as scalars) and list contexts (when we treat them as arrays and hashes).

Many things in Perl are very specific about which context they require and will force lists into scalar context when required. For example the + (plus or addition) operator expects that its two arguments will be scalars. Hence:

```

0 my @a = (1,2,3);
1 my @b = (4,5,6,7);
2 print @a + @b;

```

will print 7. There's also a third type of context, the null context, where the result of an operation is just thrown away. This usually isn't discussed, because by its very definition we don't care about what result is returned

- **Lists:** A list is an unnamed array.

```

0 print ("Hello", ",", " ", "World", "\n");

```

If you come across something that wants a LIST, you can either give it the elements of list as in the first example above, or you can pass it an array by name. If you come across something that wants an ARRAY, you have to actually give it the name of an array. Examples of functions which insist on wanting an ARRAY are push() and pop(), which can be used for adding and removing elements from the end of an array

- **Anonymous array reference:**

```
0 my $ref = [1,2,3];
```

The `[ ... ]` syntax is used to create and assign an anonymous array reference. Square brackets `[ ]` create a memory reference to an array that has no name. This reference is a single scalar value, meaning it is stored in a standard scalar variable (`$`).

- **Multiple dimensional arrays:**

```
0 my $matrix = [  
1     [1,2,3],  
2     [4,5,6],  
3     [7,8,9]  
4 ];
```

Passing a reference to a subroutine avoids flattening your data structure and is much faster for large lists because Perl only copies the single memory address rather than the entire list.

- **Dereferencing:**

```
0 print $array_ref->[0]; # Outputs: apple  
1  
2 # Loop through all elements  
3 for my $item (@$array_ref) {  
4     print "$item\n";  
5 }
```

The syntax

```
0 @{$array_ref}
```

is used to dereference an array reference.

Hashes

- **Hashes:** A hash is a two-dimensional array which contains keys and values, they're sometimes called "associative arrays", or "lookup tables". Instead of looking up items in a hash by an array index, you can look up values by their keys.

Hashes are initialised in exactly the same way as arrays, with a comma separated list of values:

```
0 my %monthdays = ("January", 31, "February", 28, "March", 31,  
  ↪ ...);
```

Or,

```
0 my %monthdays = (  
1   "January" => 31,  
2   "February" => 28,  
3   "March" => 31,  
4   ...  
5 );
```

The => operator is syntactically the same as the comma, but is used to distinguish hashes more easily from normal arrays. It's pronounced "fat comma", or less often "fat arrow". It does have one difference, you don't need to put quotes around a bare word immediately before the => operator as these are always treated as strings:

- **Reading hash values:**

```
0 print $monthdays{"January"}; # prints 31
```

Bare words inside the braces of the hash-lookup will be interpreted in Perl as strings, so usually you can drop the quotes:

- **Creating hash elements:** You can also create elements in a hash on the fly:

```
0 my %monthdays = ();  
1 $monthdays{"January"} = 31;  
2 $monthdays{February} = 28;  
3 ...
```

- **Changing hash values:** To change a value in a hash, just assign the new value to your key:

```
0 $pizza_prices{small} = '$6.00'; # Small pizza prices have  
  ↪ gone up
```

- **Deleting hash values:** Use the delete function

```
0 delete($pizza_prices{medium}); # No medium pizzas anymore.
```

- **Finding out the size of a hash:** Strictly speaking there is no equivalent to using an array in a scalar context to get the size of a hash. If you take a hash in a scalar context you get back the number of buckets used in the hash, or zero if it is empty. This is only really useful to determine whether or not there are any items in the hash, not how many.

If you want to know the size of a hash, the number of key-value pairs, you can use the `keys` function in a scalar context. The `keys` function returns a list of all the keys in the hash.

```
0 my $size = keys %monthdays;  
1 print $size;
```

Special variables

- **Intro:** Perl has many special variables. These are used to set or retrieve certain values which affect the way your program runs. For instance, you can set a special variable to turn interpreter warnings on and off (`$^W`), or read a special variable to find out the command line arguments passed to your script (`@ARGV`).
- `$_`: Represents the current thing that your Perl script's working with — often a line of text or an element of a list or hash. It can be set explicitly, or it can be set implicitly by certain looping constructs (which we'll look at later).

The special variable `$_` is often the default argument for functions in Perl. For instance, the `print()` function defaults to printing `$_`.

```
0  $_ = "Hello, world!\n";
1  print;
```

- **@ARGV:** Command line args end up in this special array.

```
0  # ./main.pl arg1 arg2
1  print "@{ARGV}" arg1 arg2
2  print my $argc = @ARGV; 2
```

- **%ENV:** This hash contains the names and values of environment variables.
- **!:** `!` contains the operating-system error corresponding roughly to `errno` in C.

```
0  open(my $fh, "<", "foo.txt")
1      or die "open failed: !";
```

- **Input record separator `$/`:** Normally,

```
0  $/ = "\n";
```

- **Proc ID `$$`:** When used by itself without a variable name, `$$` is a predefined global variable that returns the Process ID (PID) of the currently running Perl script.
- **List of general special variables:**
 - `$_` Default input / working variable
 - `@_` Arguments passed to the current subroutine
 - `$"` Separator used when interpolating arrays into strings
 - `$$` Current process ID
 - `$0` Name of the running Perl program
 - `$(` Real group ID
 - `$)` Effective group ID
 - `$<` Real user ID

- \$> Effective user ID
- \$; Multidimensional array subscript separator; largely obsolete
- \$a, \$b Special comparison variables used by sort
- %ENV Process environment variables
- \$] Perl version as a numeric value
- \$^V Perl version as a version object
- \$^O Operating system Perl was built for
- \$^X Perl executable used to start the program
- \$^T Program start time
- \$^F Maximum system file descriptor
- \$^I In-place editing extension (-i)
- \$^M Emergency memory pool
- @F Autosplit fields when using -a
- @INC Module/library search paths
- %INC Files/modules already loaded
- \$INC Current @INC hook index in newer Perl versions
- @ISA Parent classes of a package
- %SIG Signal handlers
- %{\^HOOK} Hooks for certain Perl operations
- \${^MAX_NESTED_EVAL_BEGIN_BLOCKS} Limit for nested eval/BEGIN processing

- **Regex variables**

- \$1, \$2, \$3, ... Captured groups
- @{\^CAPTURE} All capture-buffer contents
- \$& Entire matched string
- \$' Text before the match
- \$' Text after the match
- \$+ Highest-numbered capture that matched
- \$^N Most recently closed capture
- @- Starting offsets of match/captures
- @+ Ending offsets of match/captures
- %- Named captures, including multiple captures with the same name
- %+ Named captures
- \${\^MATCH} Entire /p match
- \${\^PREMATCH} Text before a /p match
- \${\^POSTMATCH} Text after a /p match
- \${\^LAST_SUCCESSFUL_PATTERN} Last successful regex pattern
- \$^R Result of regex (?{ ... }) evaluation
- \${\^RE_COMPILE_RECURSION_LIMIT} Regex compile nesting limit
- \${\^RE_DEBUG_FLAGS} Regex debugging flags
- \${\^RE_TRIE_MAXBUF} Regex trie optimization memory control

- **Command-line and filehandle variables:**
 - @ARGV Command-line arguments
 - \$ARGV Current filename being processed by <>
 - ARGV Special filehandle used by <>
 - ARGVOUT Output filehandle used by in-place editing
 - \$. Current input line number
 - \$/ Input record separator
 - \$ Output record separator
 - \$, Output field separator
 - \${^LAST_FH} Last filehandle read
- **Error and process-result variables:**
 - \$! OS/C-library error (errno)
 - %! Symbolic access to errno values
 - \$@ Exception from the most recent eval
 - \$? Exit/wait status of the last external process
 - \$^E Extended OS-specific error information
 - \${^CHILD_ERROR_NATIVE} Native child-process status
 - \$^S Interpreter/evaluation state
 - \$^W Global warning flag
 - \${^WARNING_BITS} Lexically enabled warning categories

More on strings

- **Foldcase:** case folding is performed using the built-in `fc` function (introduced in Perl 5.16). Case folding converts strings into a canonical format meant for case-insensitive comparisons, handling complex Unicode character mappings.

```
0 use feature 'fc'; # or use v5.16;
```

```
0 $a = 'H';  
1 $b = "h";  
2  
3 print fc($a) cmp fc($b); # 0
```

- **String escape sequences:**
 - `\u` (Uppercase next): Capitalizes only the very next character.
 - `\l` (Lowercase next): Lowers only the very next character.
 - `\U` (Uppercase all): Capitalizes all characters that follow it.
 - `\L` (Lowercase all): Lowers all characters that follow it.
 - `\F` (Foldcase all): Case-folds all characters that follow it (ideal for Unicode case-insensitive matching).
 - `\E` (End modification): Stops the effect of `\U`, `\L`, or `\F`.

Operators and functions

- **Operators**

- **Arithmetic:** +, -, *, /, %, ** (exp), +=, -=, *=, /=, %, **=

Note: In fact, the shortcut notation used in += and its siblings can be used for just about any operator.

We also have post/pre increment/decrement.

- **Logical:** &&, and, ||, or, !, not, xor, //

Note: The versions and, or, not, xor have extremely low precedence compared to &&, ||, and !, lower even than the assignment operator =.

The // operator is defined-or, which returns the left side if it is defined, even if it is false (like 0 or "").

- **Numeric conditional:** ==, !=, <, >, <=, >=, <=>
- **String conditional:** eq, ne, lt, gt, le, ge, cmp

Note: <=> and cmp perform three-way sorting and evaluation. They return -1, 0, or 1 for less, equal, or greater.

- **String:** . (concat), x (repeat)

```
0  "hello" . "world";
1  "hello" x 3;
```

- **Ternary:** Uses C syntax

```
0  condition ? if_true : if_false
```

- **Bitwise:** &, |, ^ <<, >>

- **Function calling:** Perl does not insist that functions enclose their arguments within parentheses. Both print "Hello"; and print("Hello"); are correct.

- **Argument types:**

- **SCALAR** – Any scalar variable, for example: 42, "foo", or \$a.
- **EXPR** – An expression (possibly built out of terms and operators) which evaluates to a scalar.
- **LIST** – Any named or unnamed list (remember that a named list is an array).
- **ARRAY** – A named list; usually results in the array being modified.
- **HASH** – Any named or unnamed hash.
- **PATTERN** – A pattern to match on — we'll talk more about these later on, in Regular Expressions.
- **FILEHANDLE** – A filehandle indicating a file that you've opened or one of the pseudo-files that is automatically opened, such as STDIN, STDOUT, and STDERR.

- **Data conversion functions:**

- **hex:** Interprets a string as a hexadecimal number and converts it to decimal.
- **oct:** Interprets a string as an octal number and converts it to decimal. If the string starts with 0x, it automatically parses it as hex, or with 0b as binary.
- **int:** Strips the decimal portion of a floating-point number, truncating it to an integer.
- **chr:** Converts an ASCII/Unicode numeric value into its corresponding character.
- **ord:** The inverse of chr. Converts the first character of a string into its numeric ASCII/Unicode value.
- **join:** Flattens a list or array into a single string, using a specified separator string.

```
0 my $string = join("-", ("a", "b", "c")); # Returns  
↪ "a-b-c"
```

- **split:** The inverse of join. Breaks a string into a list of substrings using a regular expression delimiter.

```
0 my @array = split(/-/ , "a-b-c"); # Returns ('a', 'b',  
↪ 'c')
```

9.1 Quote operators and siblings

- **q// (Single Quote):** Behaves exactly like standard single quotes ('...'). It does not interpolate variables or escape sequences.
- **qq// (Double Quote):** Behaves exactly like standard double quotes ("..."). It interpolates variables, math, and escape sequences (like \n or \U).
- **qw// (Quote Words):** Splits a string on whitespace and returns a list of literal words. No variable interpolation occurs

9.2 Important builtin functions

- **Numeric:**

- `abs($x)` Absolute value
- `sqrt($x)` Square root
- `exp($x)` $e^{**} \$x$
- `log($x)` Natural logarithm
- `sin($x)` Sine
- `cos($x)` Cosine
- `atan2($y, $x)` Arctangent
- `rand($n)` Random number from 0 to $< \$n$
- `srand()` Seed random number generator

- **String:**

- `length($str)` String length
- `substr($str, ...)` Extract/replace substring
- `index($str, $sub)` Find substring
- `rindex($str, $sub)` Find substring from right
- `lc($str)` Lowercase entire string
- `uc($str)` Uppercase entire string
- `fc($str)` foldcase entire string
- `lcfirst($str)` Lowercase first character
- `ucfirst($str)` Uppercase first character
- `chomp($str)` Remove input record separator, usually newline
- `chop($str)` Remove final character
- `sprintf(...)` Produce formatted string

```
0  sprintf(FORMAT, LIST);  
1  
2  my $price = 19.9962;  
3  my $money = sprintf("%.2f", $price);
```

- `split(...)` Split string into list
- `join(...)` Combine list into string

- **Arrays:**

- `push(@a, $x)` Add to end
- `pop(@a)` Remove from end
- `unshift(@a, $x)` Add to beginning
- `shift(@a)` Remove from beginning
- `splice(@a, ...)` Insert/remove array elements
- `reverse(@a)` Reverse elements
- `sort(@a)` Sort elements
- `grep ... @a` Filter elements
- `map ... @a` Transform elements
- `join($sep, @a)` Convert array to string
- `scalar(@a)` Number of elements
- `each(@a)` Iterate index/value pairs on modern Perl

```

0  my @nums = (3, 1, 2);
1
2  push @nums, 4;
3
4  my $last = pop @nums;
5
6  my @sorted = sort { $a <=> $b } @nums;
7
8  my @even = grep { $_ % 2 == 0 } @nums;
9
10 my @doubled = map { $_ * 2 } @nums;

```

- **Hashes:**

- keys(%h) Get keys
- values(%h) Get values
- each(%h) Get key/value pairs while iterating
- exists(\$h{\$key}) Check whether key exists
- delete(\$h{\$key}) Remove key
- scalar(keys %h) Number of keys

Constructs

- If

```
0  if (cond) {
1      ...
2  }
3
4  elsif (cond) {
5      ...
6  }
7
8  else {
9      ...
10 }
```

- Unless: inverse of an if

```
0  unless (cond) {
1      ...
2  }
```

Note that the brackets are required, and there is no `elsunless`.

- Shorthand: If if or unless require a single statement, we can put the statement before the check

```
0  print "Hello" if 1;
```

- Existence and definitiveness: To find out if something is defined, use Perl's defined function. The defined function is necessary to distinguish between a value that is false because it is undefined and a value that is false but defined, such as 0 (zero) or "" (the empty string).

```
0  my $skippy; # $skippy is undefined and false
1  $skippy = "bush kangaroo"; # true and defined
2  print "true" if $skippy; # prints true
3  print "defined" if defined($skippy); # prints defined
4
5  my $undef = undef;
6  print "defined" if defined($undef); # does not print
7  print "undefined" if not defined($undef); # prints undefined
```

- undef: When you pass a variable or expression to it, undef behaves as a function that un-defines the variable and frees its memory. The expression must be an lvalue

```
0  undef $scalar;          # Resets a scalar to the undefined state
```

When used on its own without any arguments, it simply returns the undefined value.

```
0 my $name = undef;    # Explicitly assigning the undefined
  ↪ value
```

- **While:**

```
0 while (cond) {
1     ...
2 }
```

- **Until:** The opposite of a while

```
0 until (cond) {
1     ...
2 }
```

- **Shorthand while / unless:** Similar to if and unless,

```
0 print "Hello" while 1;
1 print "Hello" until 0;
```

- **For:** Identical to C

```
0 for (my $i = 0; $i < $n; ++$i)
```

- **Foreach:**

```
0 foreach(@arr) {
1     print "$_\n";
2 }
3
4 print "${_}\n" foreach(@arr);
```

Note: When in a foreach loop, the variable representing the current element is the current element, not a copy of it.

- **Foreach range syntax:**

```
0 print "${_}" foreach (1..10)
```

- **For / foreach and variable creation**

```

0  for my $i (0..10) { ... }
1  foreach my $i ((1,2,3)) {...}

```

- **Note about for and foreach:** In Perl, you can iterate through an array using the for loop (or foreach loop). Perl treats for and foreach as completely interchangeable synonyms.

```

0  my @arr = qw(1 2 3);
1  print "$_\n" for @arr;

```

- **Postfix for:** In Perl, the postfix for modifier (also known as a statement modifier) allows you to place a loop after a single statement

```

0  STATEMENT for LIST;

```

```

0  $_ *= 2 for @nums

```

- **= do ...:**

```

0  my $variable = do { ... };

```

executes a block of code and returns the value of the last statement evaluated inside that block

- **eval BLOCK:**

```

0  eval {...}

```

acts like a try-catch block.

```

0  # Example of trapping a division by zero error
1  eval {
2      my $result = 10 / 0;
3  };
4
5  if ($?) {
6      print "Error caught: $@";
7  }

```

- **eval EXPR:** This form compiles code string text dynamically

```

0  # Example of dynamically executing a string
1  my $code = '$x = 5 + 5;';
2  eval $code;
3  print $x; # Outputs 10

```

- **Labeled blocks:** You can assign a name (called a label) to any code block by placing the name followed by a colon (:) right before the opening brace. This allows you to jump out of or control specific nested blocks using loop control statements like `last`, `next`, or `redo`.

Always write labels in ALL CAPS to ensure they stand out and never conflict with reserved Perl keywords.

```

0  OUTER_LOOP: while ($x < 10) {
1      INNER_LOOP: while ($y < 10) {
2          if ($condition) {
3              # Exits BOTH loops completely, not just the inner
3              ↪ one
4              last OUTER_LOOP;
5          }
6          $y++;
7      }
8      $x++;
9  }

```

- `redo` restarts the current iteration immediately.
- `next` skips to the next iteration
- `last` exits the loop completely (like `break` in C/Java).
- **BEGIN:** A BEGIN block in Perl is a special type of code block that runs as soon as it is parsed and compiled, before the rest of the surrounding program runs.
 - **Timing:** It executes immediately during the compilation phase, even before the rest of the script or file is fully compiled.
 - **Order:** Multiple BEGIN blocks run in the order they appear in your code.
 - **Purpose:** Commonly used to load modules (use is implemented using BEGIN), define subroutines, or set up global configuration variables before other code compiles.
 - **Automatic Deletion:** Once a BEGIN block has run, it is immediately deleted from memory for the rest of the program

```

0  print "2. This runs during runtime.\n";
1
2  BEGIN {
3      print "1. This runs first, during compilation.\n";
4  }

```

```

0  1. This runs first, during compilation.
1  2. This runs during runtime.

```

- **END:** END {} block in Perl runs code automatically just before the Perl interpreter exits the program.
 - **Timing:** It runs as late as possible during program shutdown, acting like a destructor.
 - **Order:** Multiple END blocks run in the reverse order of how they appear in your code.
 - **Exceptions:** END blocks run even if the program exits using die(). They do not run if the program exits via exec or a hard operating system signal like SIGKILL.
 - **Common Uses:** They are great for cleanup tasks, such as closing file handles, removing temporary files, or disconnecting from databases.
- **__DATA__:** __DATA__ token tells the compiler that the compilation of executable code is finished. Everything written below this token is treated as data, which Perl automatically makes accessible to your program through a special, built-in filehandle named DATA.

```

0  use strict;
1  use warnings;
2
3  # Loop through each line in the __DATA__ section
4  while (my $line = <DATA>) {
5      chomp($line);
6      print "Processing line: $line\n";
7  }
8
9  __DATA__
10 First line of data
11 Second line of data
12 Third line of data

```

- **Heredocs:**

```

0  # 1. Double-quoted (Interpolated)
1  print <<"EOF";
2  Hello, $name!
3  This is a multi-line string.
4  EOF

```

- **Backtick heredoc:**

```

0  my $output = <<`END_CMD`
1  ls -la /tmp
2  echo "----"
3  df -h /
4  echo "----"
5  uptime
6  END_CMD

```

That's a backtick heredoc. Everything between the markers executes as a shell command. The output comes back as a string. Multi-line shell scripts, captured in one clean shot.

- **Heredoc variants:**

- «"TAG" Interpolates variables
- «'TAG' No interpolation (literal)
- «`TAG` Executes as shell command

- **Stacked heredocs:**

```
0 my ($sql, $html) = (<<'SQL', <<'HTML');
1 SELECT name, email
2 FROM users
3 WHERE active = 1
4 ORDER BY name
5 SQL
6 <;div class="results">;
7     <;h2>;Active Users<;/h2>;
8     <;ul id="user-list">;<;/ul>;
9 <;/div>;
10 HTML
```

- **goto:** Three forms

```
0 goto LABEL
1 goto EXPR
2 goto &NAME
```

- **goto LABEL:** This form instantly jumps to the line marked with a specific label name followed by a colon (LABEL:).

You cannot use this to jump into a construct requiring initialization, such as a subroutine or a foreach loop. It also cannot be used to jump out of a block passed to sort.

- **goto EXPR:** This evaluates an expression at runtime to get a string, and then jumps to the label matching that string. This behaves like a computed goto found in languages like Fortran

```
0 my @targets = ("FOO", "BAR", "BAZ");
1 my $index = 1;
2
3 goto $targets[$index]; # Evaluates to "BAR" and jumps
   ↪ there
4
5 FOO: print "Hit FOO\n";
6 BAR: print "Hit BAR\n"; # Execution jumps here
7 BAZ: print "Hit BAZ\n";
```

- **goto &NAME**: This is a specialized, "highly magical" feature that doesn't carry the standard stigma of goto. Instead of jumping to a text label, it exits the current subroutine and immediately calls the new subroutine in its place.

The new subroutine completely inherits the argument stack (@_) of the old subroutine. When the new subroutine finished executing, it returns control directly back to whoever called the first subroutine

```
0  sub first_sub {  
1      print "Inside first_sub\n";  
2      goto &second_sub; # Exits first_sub, replaces it  
    ↪ with second_sub  
3  }  
4  
5  sub second_sub {  
6      print "Inside second_sub with arguments: @_ \n";  
7      # When this ends, control goes back to wherever  
    ↪ first_sub() was called.  
8  }  
9  
10 first_sub("Hello", "World");
```

- **CHECK { ... }**: A CHECK block runs after the Perl compiler finishes compiling your code, but before the program actually starts running.

Multiple CHECK blocks run in reverse order of how they appear in your file (last-defined runs first).

- **INIT { ... }**: An INIT block runs right before the Perl runtime system begins executing your main program.

Just after compilation and any CHECK blocks finish, but right before the main script starts.

Multiple INIT blocks run in normal order (first-defined runs first).

Intro to Perl IO

- **Predefined file handles:**

```
0 STDIN    # standard input
1 STDOUT   # standard output
2 STDERR   # standard error
```

- **<...> syntax:** Primarily a read operation on a filehandle

```
0 my $line = <STDIN>;
```

Read the next line from standard input.

- **Writing to file handle:**

```
0 print STDOUT "Hello\n";
```

Although STDOUT is the default.

- **readline:** Reads a line from a file handle

```
0 my $line = readline(STDIN);
```

Reads a line from

- **say and printf:** Say is essentially C puts; takes a single string argument and appends a newline before printing it.

```
0 use feature "say";
1 say "Hello, world";
2 printf "%s", "Hello, world";
```

- **Opening a file:**

```
0 open(HANDLE, MODE, PATH);
```

- < read
- > write, truncate existing file
- » append
- +< read and write (starts at beginning)
- +> read and write, truncate
- +» read and append (starts at end)

- **Two argument open syntax:** The mode and the file path are combined into a single string

```
0  open(HANDLE, "MODE PATH")
```

- **Closing a file:**

```
0  close(HANDLE);
```

- **Empty diamond:** The empty diamond operator reads from all files specified on the command line (in order).

```
0  while (<>) {  
1    print;  
2  }
```

- **Slurp mode:** We can temporarily make the input separator undefined, then read a file.

```
0  open(my $file, "<", "file.txt")  
1    or die "$!";  
2  
3  my $contents = do {  
4    local $/  
5    <$file>;  
6  }
```

This is called **file slurping**

11.1 File IO

- **Line input operator in list context:**

```
0  # reading lines from STDIN (or from files on the command
    ↪ line)
1  while (<>) {
2      # Process the line of input in $_
3  }
4
5  # reading a single line of user input from STDIN
6  my $input = <STDIN>;
7
8  # reading all lines from STDIN into an array
9  my @input = <STDIN>;
```

In scalar context, the line input operator yields the next line of the file referenced by the filehandle given.

In list context, the line input operator yields all remaining lines of the file referenced by the filehandle. (Be careful when using this as you may use up all your memory if the file is large).

- **Failure:** You should always check for failure of an `open()` statement:

```
0  open(LOGFILE, "< /var/log/httpd/access.log")
1      or die "Can't open /var/log/httpd/access.log: $!";
```

Attempting to read from or write to an unopened file may cause unexpected results.

- **The die function (die()):** die is a Perl function which takes an error message and terminates the program displaying that message to the user.

`$!` is the special variable which contains the error message produced by the last system interaction.

Perl tries to be helpful when dying on errors and will append the appropriate filename and line number of your script to the end of the die message, with a newline. If you don't want this behaviour, end the die message with a newline (`\n`) character.

```
0  # The following provides an error with file and line-number:
1  open(LOGFILE, "< $file") or die "Cannot open $file: $!";
2
3  # Here the file and line-number are omitted.
4  open(LOGFILE, "< $file") or die "Cannot open $file: $!\n";
```

- **Fatal:** An alternative to explicitly checking whether `open` and other functions succeeded is to use the `Fatal` module:

```

0  use Fatal qw(open close);
1
2  open(LOGFILE, "< $file");    # no need to check!
3
4  close(LOGFILE);             # no need to check!

```

The Fatal module creates its own functions for the ones you have passed in, and tells Perl to use those instead. These throw an exception (die) if the original function returns a false value. Care should be taken when retro-actively using Fatal on existing programs. It changes the behaviour of the specified functions for the whole program, not just for your file.

- **Filehandles:** The first argument we pass to open is a filehandle. We can use this to have access to the file for the mode in which it was opened.

```

0  # Open access.log for reading using LOGFILE as our filehandle
1  open(LOGFILE, "<", "/var/log/httpd/access.log")
2      or die "Can't open /var/log/httpd/access.log: $!";
3
4  # use the filehandle in the <> line input operator to read
   ↪ the contents
5  while (<LOGFILE>) {
6      print if /perltraining.com.au/;
7  }
8  close LOGFILE;
9
10 # open a new logfile for appending
11 open(SCRIPTLOG, ">>", "myscript.log") or die "Can't open
   ↪ myscript.log: $!";
12
13 # print() takes an optional filehandle argument - defaults to
   ↪ STDOUT
14 print SCRIPTLOG "Opened logfile successfully.\n";
15
16 close SCRIPTLOG;

```

Note that you should always close a filehandle when you're finished with it (even though any open filehandles will be automatically closed when your script exits).

- **Scalar filehandles:** Under Perl version 5.6.0 and above, you can provide a scalar as the first argument to the open function. This means that your filehandles can have scope, and makes it easier to pass them to subroutines and put into structures such as hashes and arrays. Where possible it is a good idea to always use scalar filehandles.

```

0  my $fh;
1  open($fh, "<", "/path/to/file") or die $!;
2
3  # We can also declare the variable inside the call to open
4  open(my $out_fh, "> /path/to/other/file") or die $!;

```

In versions before 5.6.0 you can do the same thing by using the `FileHandle` module, but you need to declare your intentions first:

```
0 use FileHandle;
1 my $fh = FileHandle->new; # $fh is now a FileHandle object.
2 open ($fh, "<", "/path/to/file") or die $!;
```

- **Changing file contents:** When manipulating files, we may wish to change their contents. A flexible way of reading and writing a file is to import the file into an array, manipulate the array, then output each element again. It is important to ensure that should anything go wrong we don't lose our original data. As a result,

it's considered best-practice to write our data out to a temporary file and then move that over the input file after everything has been successful.

```
0 # a program that reads in a file and writes the lines in
  ↪ sorted order.
1 use File::Copy;
2
3 open(my $infile, "<", "file.txt") or die "Can't open file.txt
  ↪ for input: $!";
4 my @lines = <$infile>; # Slurps all the lines into @lines.
5 close $infile;
6
7 @lines = sort @lines;
8
9 # open temporary file to save our sorted data into
10 open(my $outfile, ">", "file.txt.tmp") or die "Can't open
  ↪ file.txt for output: $!";
11
12 # use print's ability to print lists
13 print {$outfile} @lines;
14
15 close $outfile;
16
17 # we know that we were successful, so write over the original
  ↪ file
18 move("file.txt.tmp", "file.txt") or die "Could not rename
  ↪ temp file: $!";
```

- **Secure temporary files:** In the above example we create our own temporary file using a predictable filename. This may cause our program to over-write a temporary file that's currently in use, which is obviously a bad thing. To create and use unique temporary files we can use the `File::Temp` module:

```

0  use File::Temp qw(tempfile);
1
2  my ($tmp_fh, $tmp_name) = tempfile();
3
4  # use print's ability to print lists
5  print {$tmp_fh} @lines;
6
7  close $tmp_fh;
8
9  move($tmp_name, "file.txt") or die "Could not rename
   ↪ $tmp_name: $!";

```

The File::Temp module can also be used to create in-memory temporary files if required.

- **Open a file for reading and writing (Read + write mode):** Files can be opened for simultaneous read/write by putting a + in front of the > or < sign. +< is almost always preferable, as +> would overwrite the file before you had a chance to read from it.
 - +< (Read + Write, No Truncation)
 - * **Behavior:** Opens an existing file for both reading and writing. It does not clear the file contents when opened. The file pointer starts at the beginning of the file.
 - * **File status:** The file must already exist (or it fails depending on usage, though standard behavior expects an existing file).
 - * **Use case:** When you want to read from a file, modify specific parts, or overwrite parts of it in place without losing the rest of the data.
 - +> (Read + Write, Truncate/Clobber)
 - * **Behavior:** Opens a file for reading and writing, but deletes all existing contents (truncates the file to zero length) immediately when opened.
 - * **File status:** Creates the file if it does not exist.
 - * **Use case:** Rare; useful when you want a completely fresh scratchpad file that you can both write to and read back from during the same session.

+< puts you at the start of the file. Note that it won't create a new file if the file you're dealing with does not exist (you'll just get an error that the file doesn't exist). If you start writing before you've reached the end of the file, you will overwrite characters in that file (from that point). Even if you're dealing with fixed-length records and think you know what you're doing, this is often still a bad idea.

+» initially puts you at the end of the file. It will create a new file if necessary and will not clobber an old one. It allows you to read at any point in the file, but all writes will always go to the end.

- **Buffering:** Since both print and readline (<>) are buffered, you should not use them for editing a file in-place. If you must do so, use the lower level functions such as sysseek(), syswrite() and sysread(). Perl also has a -i switch, for more useful in-place modification of files.
- **Opening pipes:** If the filename given to open() begins with a pipe symbol (|), the filename is interpreted as a command to which output is to be piped, and if the filename ends with a |, the filename is to be interpreted as a filename which pipes input to us.

you can pass a system command to open in Perl by using the pipe character (`|`). When you place a pipe at the very end or the very beginning of the command string, Perl treats it as a program to run rather than a file to read. This is known as a piped open.

```
0  # Runs the Linux 'ls' command and lets you read the file list
1  open(MY_PIPE, "ls -la |") or die "Failed: $!";
2
3  # Sends text directly into the Linux 'mail' command to send
   ↪ an email
4  open(MY_PIPE, "| mail -s 'Hello' user@example.com") or die
   ↪ "Failed: $!";
```

- Safer pipes

```
0  # Pipe output to us
1  open(my $pipe, "-|", "ls", "-la");
2
3  # Pipe our input to sort
4  open(my $pipe, "|-", "sort");
```

- Piping input example

```
0  # Open our external sort command.
1  open (my $sort_fh, "|sort") or die "Can't open pipe: $!";
2
3  # Our friends will be printed in sorted order.
4  foreach my $friend (qw/Jacinta Damian Kirrily Paul/) {
5      # This does not print to stdout, gives the names to sort
       ↪ one at a time.
6      print {$sort_fh} "$friend\n";
7  }
8
9  # Closing the pipe handle causes sort to cease thus giving
   ↪ output
10 close $sort_fh;
```

- Piping output example

```

0  #!/usr/bin/perl -w
1  # This program allows us to read a file from another machine
2  # using secure shell. This is most useful if we can login
   ↳ without
3  # a password (eg, established keys).
4  use strict;
5
6  # Process our command line arguments, and complain if we
   ↳ don't
7  # have both a host and filename.
8  my ($host, $file) = @ARGV;
9  unless ($host and $file) {
10     die "Usage: $0 host filename\n";
11 }
12
13 open (SSH, "ssh $host cat $file |") or die "Can't open pipe:
   ↳ $!";
14
15 while(<SSH>) {
16     # We can process the file in any way we like here.
17     # In this particular case, we'll simply print it to
18     # our STDOUT.
19     print;
20 }

```

- **File locking:** File locking can be achieved using the flock() function. This can be used to guard against race conditions or other problems which occur when two (or more) processes want to access the same file at the same time.

flock is Perl's portable file-locking mechanism, and works on most operating systems (and produces a fatal error on those which it does not). The locks set by flock are advisory only, which means that a process that chooses not to use flock can (and will) ignore any locks in place. flock can only lock entire files, not individual records. Depending upon your setup, flock may or may not work over NFS.

```

0  # using flock
1  use Fcntl ':flock'; # import LOCK_* constants
2
3  flock(FILEHANDLE, LOCK_EX); # exclusive (write) access
4  flock(FILEHANDLE, LOCK_SH); # shared (read-only) access

```

As flock only works on filehandles, instead of filenames, you have to open the file first before you try to lock it. It's important to make sure that you open the file for writing, if you intend to write to it, and that you don't clobber the contents of the file when doing so. This is a good use of +<. Closing a locked file releases any locks the process holds upon it. This is good because it means that if your process exits unexpectedly all locks it held are released and other processes may then go forward with their locks.

```

0  use Fcntl ':flock'; # import LOCK_* constants
1
2  # Open file for read and write
3  open my $file_fh, "+<", $file or die "Failed to open $file:
   ↳ $!";
4
5  # Lock the file for writing (exclusive lock) make sure it
   ↳ worked.
6  flock($file_fh, LOCK_EX) or die "Failed to gain lock on
   ↳ $file: $!";
7
8  # At this point we have exclusive access to the file.
9  # Wipe previous process' details
10 truncate($file_fh, 0) or die "Failed to truncate $file: $!";
11
12 # Write to the file, or perform other operations as needed
   ↳ here...
13 print {$file_fh} $data;
14
15 close $file_fh; # Closing the file releases the lock as well

```

flock will wait indefinitely until the lock is granted, however it can return early if interrupted by a signal or other event. It's important to ensure that flock returns true to be sure that you have the lock you requested. It is possible to make flock non-blocking as follows:

```

0  use Fcntl ':flock'; # import LOCK_* constants
1
2  flock(FILEHANDLE, LOCK_EX | LOCK_NB); # non-blocking
   ↳ exclusive lock
3  flock(FILEHANDLE, LOCK_SH | LOCK_NB); # non-blocking shared
   ↳ lock

```

All attempts to get a non-blocking lock return immediately with either true for success (the lock was obtained) or false for failure (the lock was not obtained).

- **Handling binary data / read():** If you are opening a file which contains binary data, you probably don't want to read it in a line at a time using while (<>) { }, as there's no guarantee that there will be any line breaks in the data, and we'll probably end up with very uneven chunks.

Instead, we can use read() to read a certain number of bytes from a file handle. However, before we do that, we should call the binmode() function on the filehandle, so that Perl knows that we'll be dealing with a binary file. This means Perl won't try to do any transformations of input based upon the operating system or locale where your program is running.

binmode() must be called on the filehandle before any corresponding file I/O. It's best to call it immediately after you open the file.

`read()` takes the following arguments:

- The filehandle to read from
- The scalar to put the binary data into
- The number of bytes to read
- The byte offset to start from (defaults to 0)

11.2 Directory Interaction

- **The globbing operator:** If the angle brackets have anything in them other than a filename or nothing, it will work as a globbing operator and whatever is between the angle brackets will be treated as a filename wildcard. For instance

```
0 my @files = <*.txt>
```

The filename glob `*.txt` is matched against files in the current directory, then either they are returned as a list (in list context, as above) or one scalar at a time (in scalar context).

```
0 while (my $filename = <*.txt>) {  
1     open (my $in_fh, "<", $filename) or die ("Can't open  
    ↪ $filename: $!");  
2  
3     # Read from the file  
4     close $in_fh;  
5 }
```

The `glob()` function behaves in a very similar manner to the angle bracket globbing operator.

```
0 my @files = glob("*.txt");  
1 foreach my $file (glob("*.txt")) {  
2     # Process $file  
3 }
```

The `glob()` is considered much cleaner and better to use than the angle-brackets globbing operator.

When using `glob`, you can combine more than one pattern in order to get a wider selection of files. For example, to get all the `.txt` and `.pl` files you can write:

```
0 glob("*.txt *.pl");
```

- **File test operators**
 - `-e` File exists.
 - `-r` File is readable
 - `-w` File is writable
 - `-x` File is executable
 - `-o` File is owned by you
 - `-z` File has zero size.
 - `-s` File has nonzero size (returns size).

- -f File is a plain file (as opposed to a directory, symbolic link, device etc.)
- -d File is a directory.
- -l File is a symbolic link.
- -p File is a named pipe (FIFO), or Filehandle is a pipe.
- -S File is a socket.
- -b File is a block special file.
- -c File is a character special file.
- -t Filehandle is opened to a tty.
- -u File has setuid bit set.
- -g File has setgid bit set.
- -k File has sticky bit set.
- -T File is a text file.
- -B File is a binary file (opposite of -T).
- -M Days since file last modified, when script started.
- -A Same for access time.
- -C Same for inode change time.

```

0  #!/usr/bin/perl -w
1  use strict;
2  unless (-e "config.txt") {
3      die "Config file doesn't exist";
4  }
5
6  # or equivalently...
7  die "Config file doesn't exist" unless -e "config.txt";

```

- **Finding information about files:** We can find out various information about files by using file test operators and functions such as stat().

In Perl, the built-in stat() function returns a 13-element list containing detailed status information and metadata for a specific file, directory, or filehandle

stat() a filename expression, a filehandle, or a directory handle. If you leave the argument blank, it defaults to evaluating the global variable \$_.

```

0  # Capturing all 13 elements into distinct scalar variables
1  my ($dev, $ino, $mode, $nlink, $uid, $gid, $rdev, $size,
2      $atime, $mtime, $ctime, $blksize, $blocks) =
    ↪ stat($filename);

```

When stat() is successful, it returns the following fields in this exact sequence:

Index	Element	Description
0-6	\$dev - \$rdev	Device, Inode, Mode, Links, UID, GID, Rdev
7	\$size	Total size of the file in bytes
8-10	\$atime, \$mtime, \$ctime	Access, Modify, Change times (Epoch seconds)
11-12	\$blksize, \$blocks	Block size and count

Note: If `stat()` fails, it returns an empty list `()` in list context or `0` in scalar context.

- **Multiple file tests:** Perl caches the result of file tests in a special filehandle called `__` (underscore). Performing tests on this filehandle can often avoid subsequent system calls, resulting in a slight performance gain.

```
0  if(-r $file && -w _) {
1      # ...
2  }
```

- **Changing the working directory:** The function `chdir` allows you to change your program's working directory. All relative file access from that point on will use the new directory. Note that this does not change the working directory of the calling process

```
0  # Archive log files
1  my $tar = "/bin/tar";
2  my $date = "01-01-00";
3  my $directory = "/var/log/apache/";
4
5  unless( chdir($directory) ) {
6      die "Failed to chdir to $directory: $!";
7  }
8
9  # Get all files in this directory
10 my @files = glob("*.log*");
11 system("$tar -czf weblogs.$date.tgz @files") if @files;
```

- **CWD module:**

To find out what your current working directory is, we can use the `Cwd` module:

```
0  use Cwd;
1  my $current_working_directory = getcwd();
```

- **Recurring down directories:** The built-in functions described above do not enable you to easily recurse through subdirectories. Luckily, the `File::Find` module is part of the standard library distributed with Perl 5.

File::Find emulates Unix's find command. It takes as its arguments a subroutine to execute for each file found, and a list of directories to search. Note that to pass a reference to a subroutine we prefix the name of the subroutine with \&

```
0  #!/usr/bin/perl -w
1  use strict;
2  use File::Find;
3
4  print "Enter the directory to start searching in: ";
5  chomp(my $dir = <STDIN>);
6
7  # find takes a subroutine reference and the directory to
   ↪ start working from.
8  find (\&wanted, $dir);
9
10 sub wanted {
11     if(/\..pl$/) { # See if it's a .pl file
12         print "$File::Find::name\n"; # Print the current file
           ↪ name.
13     }
14 }
```

For each file found, certain variables are set.

- \$__ is set to the name of the current file.
- \$File::Find::dir is set to the directory that contains the file.
- \$File::Find::name contains the full name of the file, i.e. \$File::Find::dir/\$__.

File::Find automatically changes your current working directory to the same as the file you are currently examining. Thus there's rarely a need to use \$File::Find::dir. If all you want to do is process the file regardless of its location on the file system you can simply open the file using the name in \$__.

- **opendir and readdir:** We can also open directories for access by using the opendir() function. Once a directory is open, we can read file names from it using the readdir() function.

To read the contents of files in the directory, you still need to open each one using the open() function.

```
0  # $ENV{HOME} stores the home directory on Unix platforms, use
1  # $ENV{HOMEPATH} for MS Windows
2
3  opendir(HOMEDIR, $home) or die "Can't read dir $home: $!";
4
5  my @files = readdir(HOMEDIR);
6
7  closedir HOMEDIR;
```

- **Scalar directory handles:** Under Perl 5.6.1 and above you can provide a scalar as the argument to readdir. This allows you to have scalar directory handles which

have scope and makes it easier for you to pass them to subroutines or include them in hashes and arrays. Just like scalar filehandles, these are the preferred option if you can use them.

```
0 my $homedir;
1 opendir($homedir, $home) or die "Can't read dir $home: $!";
2 my @files = readdir($homedir);
```

- **glob and readdir:**

glob	readdir
Slower	Faster
Allows you to filter filenames	Gives you all filenames
Returns files in asciibetical order	Returns files in file-system order
Returns filename with path intact	Returns filename only
Does not return dot files when called as glob("*") (although glob("*.") does).	Returns all filenames

- **rewinddir:** We can rewind the current position of the directory handle back to the beginning by using the rewinddir function.

```
0 rewinddir HOMEDIR;
```

Unfortunately this is rarely as useful as you might at first think. `rewinddir` does not refresh the directory listing when it rewinds. To see whether the directory listing has changed since your program started you'll have to close the directory and reopen it.

Subroutines

- Creating and calling subroutines

```
0  sub fn {  
1      ...  
2  }  
3  
4  fn(); # Preferred method of calling  
5  &fn();  
6  &fn;  
7  fn;
```

The older ampersand syntax bypasses any defined prototypes and implicitly passes the caller's current `@_` argument array.

You can pass arguments to a subroutine by including them in the parentheses when you call it. The arguments end up in a special array called `@_` which is only visible inside the subroutine.

- **Shift and unshift:** The Perl shift function removes and returns the first element of an array. This shortens the array by one and shifts all subsequent elements to the left. If the target array is empty, it returns undef

The unshift function adds one or more elements directly to the beginning of an array. It is the exact opposite of the shift function, which removes an item from the front.

```
0  sub print_headers {  
1      my $title = shift "Untitled";  
2      my $author = shift "Anonymous";  
3      print "$title\n"  
4      print "by\n"  
5      print "$author\n";  
6  }
```

- **Passing arrays and hashes:** To pass in a single array or hash to a subroutine, make it the final element in your argument list.

Passing in more than one array or hash causes problems. This is because of list flattening. When Perl sees a number of items in parentheses these are combined into one big list.

```
0  # Flatten two lists into one big list and put that in an  
    ↪ array  
1  my @biglist = (@list1, @list2);  
2  
3  # Flatten (join) two hashes into one big list and put that in  
    ↪ a hash  
4  my %bighash = (%hash1, %hash2);
```

```

0  my @colours = qw/red blue white green pink/;
1  my @chosen = qw/red white green/;
2
3  print_unchosen(@chosen, @colours);
4
5  sub print_unchosen {
6      my (@chosen, @colours) = @_;
7
8      # at this point @chosen contains:
9      # (red white green red blue white green pink)
10     # and @colours contains () - the empty list.
11 }

```

We can avoid this problem by using references:

```

0  my @colours = qw/red blue white green pink/;
1  my @chosen = qw/red white green/;
2  print_unchosen(\@chosen, \@colours);
3  sub print_unchosen {
4      my ($chosen, $colours) = @_;
5      my @chosen = @$chosen;
6      my @colours = @$colours;
7      # at this point @chosen contains:
8      # (red white green)
9      # and @colours contains (red blue white green pink)
10 }

```

- **Default arguments:** Starting in Perl v5.36, subroutine signatures are stable and enabled automatically when you specify `use v5.36;` or higher. You can define default values directly inside the parameter list using `=`, `//=`, or `||=`.

```

0  use v5.36;
1
2  # 1. Standard Default (=)
3  # Triggered only if the argument is omitted entirely
4  sub greet ($name, $greeting = "Hello") {
5      say "$greeting, $name!";
6  }
7  greet("Alice");           # Hello, Alice!
8  greet("Bob", "Good morning"); # Good morning, Bob!
9  greet("Charlie", undef);  # Prints: , Charlie! (undef treated
   ↪ as a value)
10
11 # 2. Defined-or Default (//=) - Added in v5.38+
12 # Triggered if omitted OR passed as undef
13 sub set_timeout ($time //= 30) {
14     say "Timeout is $time seconds.";
15 }
16 set_timeout();           # Timeout is 30 seconds.
17 set_timeout(undef);      # Timeout is 30 seconds.
18
19 # 3. Logical-or Default (=) - Added in v5.38+
20 # Triggered if omitted, undef, or any false value (0, "")
21 sub set_status ($status = "active") {
22     say "Status: $status";
23 }
24 set_status(0);           # Status: active (0 is false)

```

Because Perl does not strictly enforce the count of elements in `@_` natively, all positional arguments are technically optional unless you explicitly check them.

```

0  sub connect_db {
1      # Unpack parameters from @_
2      my ($host, $port, $timeout) = @_;
3
4      # Assign fallback values if they aren't provided (or are
   ↪ undef)
5      $port    //= 3306;
6      $timeout //= 30;
7
8      print "Connecting to $host:$port (Timeout: $timeouts)\n";
9  }
10
11 connect_db("localhost");           # Uses defaults for port
   ↪ and timeout
12 connect_db("127.0.0.1", 8080);     # Override port, uses
   ↪ default timeout

```

- **Named arguments:** When your subroutine accepts many optional parameters, remembering the exact positional order becomes difficult. The standard Perl convention is to slurp remaining arguments into a hash, allowing users to pass named parameters

```

0  use v5.10; # For the // operator
1
2  sub create_user {
3      my ($username, %options) = @_; # First argument is
        ↪ required, rest are optional pairs
4
5      # Set fallback defaults
6      my $role   = $options{role}    // 'subscriber';
7      my $status = $options{status} // 'active';
8
9      print "Created $username as a $role ($status)\n";
10 }
11
12 # Callers can pass options in any order, or skip them
        ↪ entirely
13 create_user("jdoe");
14 create_user("msmith", role => "admin");
15 create_user("rrock", status => "pending", role =>
        ↪ "moderator");

```

- **Amperсанд fn name:** If you call a subroutine using the ampersand and omit parentheses (e.g., `&my_sub`), Perl automatically passes the caller's current argument array (`@_`)

Also, if you name a custom subroutine the same name as a built-in Perl function or keyword, the ampersand forces Perl to run your custom subroutine instead of the built-in version.

- **Subroutine references:** When paired with a backslash (`&my_sub`), it does not call the function. Instead, it creates a reference to the subroutine, which allows you to pass the function around as data

When you use parentheses, it clears the argument list.

- **Function prototypes:** In Perl, function prototypes are a mechanism used to change how the Perl parser interprets a subroutine's arguments, primarily allowing user-defined functions to mimic built-in Perl functions.

Unlike prototypes in languages like C, C++, or Java, Perl prototypes do not act as traditional function signatures. Instead of validating types at runtime, they force context coercion on variables and change how arguments are parsed at compile time

Prototypes are declared by adding special characters inside parentheses right after the subroutine name:

- `$` Forces scalar context on the argument.
- `@` Consumes all remaining arguments as a list (slurpy).

An `@` prototype is slurpy, so it must appear at the end of the prototype:

- `%` Consumes all remaining arguments as a hash.

Internally, Perl still passes a flat list:

The % prototype causes the remaining arguments to be evaluated in list context. It does not guarantee that the arguments form a valid hash, so an odd number of arguments is still possible.

It can follow mandatory arguments:

```
0  sub configure ($%) {  
1      my ($name, %options) = @_;  
2  }  
3  
4  configure(  
5      "window",  
6      width => 800,  
7      height => 600,  
8  );
```

- & Requires a code block or anonymous subroutine reference.

When & is the first prototype character, the caller can pass a bare block:

```
0  sub repeat (&$) {  
1      my ($code, $count) = @_;  
2  
3      $code->() for 1 .. $count;  
4  }  
5  
6  repeat {  
7      print "Hello\n";  
8  } 3;
```

This is approximately equivalent to:

```
0  repeat(sub {  
1      print "Hello\n";  
2  }, 3);
```

- * Accepts a typeglob. The * prototype is commonly used for filehandles:

```
0  sub write_message (*$) {  
1      my ($handle, $message) = @_;  
2      print $handle "$message\n";  
3  }  
4  
5  write_message(STDOUT, "Hello");  
6  write_message(STDERR, "An error occurred");
```

It can also accept a typeglob explicitly:

```
0 write_message(*STDOUT, "Hello");
```

The `*` prototype supports Perl's bareword filehandle syntax. It does not strictly enforce that the value is a typeglob at runtime.

- `;` Separates mandatory arguments (before `;`) from optional ones (after `;`).

```
0 sub greet ($;$) {  
1     my ($name, $greeting) = @_;  
2  
3     $greeting //= "Hello";  
4  
5     print "$greeting, $name!\n";  
6 }
```

- `\_` Requires a literal variable starting with that character and passes it by reference. It means that the caller must provide an actual variable of the corresponding kind. Perl then passes a reference to it automatically.

Note: The subroutine must be declared before its calls for the prototype to take effect.

Also, you cannot write something like

```
0 sub _display(first, @) { ... }
```

because a Perl prototype describes the context and syntactic form of each argument. It does not declare or name parameters.

The characters inside the parenthesis must be entirely prototype characters. To require one scalar argument followed by any number of additional arguments, write:

```
0 sub _display($@) { ... }
```

Modern Perl also supports actual named parameter declarations through signatures:

```
0 use v5.36;  
1 sub _display($first, @remaining) { ... }
```

Here, `($first, @remaining)` is a signature, not a prototype. It creates the lexical variables `$first` and `@remaining` automatically.

Use signatures when your goal is to name and unpack parameters. Use prototypes only when you intentionally want to alter how Perl parses or evaluates calls to the subroutine.

- **Bypassing prototype checking (ampersand prefix):** If a subroutine has a defined prototype (which restricts or formats the arguments it accepts), prefixing the call with an ampersand will completely ignore the prototype rules.
- **Anonymous functions (anonymous subroutine / subref):** . Instead of adding a name to the symbol table, it returns a code reference (CODE ref) that can be stored in a variable, passed to other functions, or executed immediately.

To create one, simply omit the name after the sub keyword. You typically assign this definition to a scalar variable:

```
0  # Defining an anonymous function and storing its reference
1  my $say_hello = sub {
2      my ($name) = @_;
3      print "Hello, $name!\n";
4  }; # Note the trailing semicolon, as this is an assignment
    ↪ statement
```

Because the variable holds a reference, you must dereference it to execute the code. There are two common ways to do this:

```
0  $say_hello->("Alice"); # Outputs: Hello, Alice!
1  &{$say_hello}("Bob"); # Outputs: Hello, Bob!
```

If you only need to run an anonymous function once right where you define it, you can enclose the definition in parentheses and invoke it immediately

```
0  (sub { print "Running immediately!\n" }->)();
```

Anonymous functions in Perl act as closures. This means they can "capture" and remember lexical variables (my) from the surrounding scope where they were created, even after that scope has exited.

```
0  sub make_multiplier {
1      my ($multiplier) = @_;
2
3      # Returns an anonymous function that retains access to
    ↪ $multiplier
4      return sub {
5          my ($number) = @_;
6          return $number * $multiplier;
7      };
8  }
9
10 my $triple = make_multiplier(3);
11 print $triple->(5); # Outputs 15
```

- **Subroutine attributes:** In Perl, subroutine attributes allow you to attach compile-time metadata or behavior modifications to your subroutines. They are specified by placing a colon (:) after the subroutine name (or signature/prototype) and listing the attributes.

Attributes can be attached to standard named subroutines, forward declarations, or anonymous subroutines:

- **lvalue:** Allows the subroutine to act as a modifiable value that can be assigned to directly (like a variable). The subroutine must return a modifiable scalar variable.

```
0  sub name :lvalue {  
1      return $name;  
2  }  
3  
4  name() = "Nathan";
```

Normally, assigning to a subroutine call is illegal. With :lvalue, Perl treats the value returned by the subroutine as the storage location to modify.

```
0  use strict;  
1  use warnings;  
2  
3  my $value = 10;  
4  
5  sub value :lvalue {  
6      $value;  
7  }  
8  
9  value() = 25;  
10  
11 print $value;      # 25  
12 print value();     # 25
```

This can be used to create clean object accessors

```
0  package Person;  
1  
2  sub new {  
3      my ($class, $name) = @_  
4  
5      return bless {  
6          name => $name,  
7      }, $class;  
8  }  
9  
10 sub name :lvalue {  
11     my ($self) = @_  
12     $self->{name};  
13 }
```

```

0 my $person = Person->new("Nathan");
1
2 print $person->name;           # Nathan

```

Perl “secretly” passes a package name when you call a subroutine using method-call syntax, most commonly with `->`.

```

0 PackageName->method(arguments);

```

is equivalent to

```

0 PackageName::method("PackageName", arguments);

```

- **method:** Explicitly marks the subroutine as an object-oriented method. It primarily tells Perl not to throw an “Ambiguous call” warning if the method shares a name with a core function
- **:prototype(...):** Assigns a prototype to the subroutine

```

0 sub fn : prototype($$) { ... }

```

It is equivalent to the older prototype placement:

```

0 sub fn($$) { ... }

```

- **:const:** `:const` turns an anonymous subroutine into a constant subroutine. Perl executes the subroutine immediately when the `sub` expression is evaluated, captures its return value, and uses that captured value for every future call.

```

0 my $number = 10;
1
2 my $get_number = sub :const {
3     $number
4 };
5
6 $number = 20;
7
8 print $get_number->();      # 10

```

The body does not run when `$get_number->()` is called. It ran earlier when the anonymous subroutine was created.

This allows more complicated initialization:

```
0  my $get_value = sub :const {  
1      my $x = 5;  
2      my $y = 7;  
3      $x * $y  
4  };  
5  
6  print $get_value->();    # 35
```

It applies only to anonymous subroutines. It cannot be placed directly on a named subroutine.

12.1 Overloading

- **Overloading:** In Perl, overloading usually means defining how objects behave when used with built-in operators such as `+`, `-`, `==`, `""`, or `0+`.

```
0  use overload
1  '+' => \&add,
2  '""' => \&to_string,
3  fallback => 1;
4
5  sub new {
6      my ($class, $x, $y) = @_;
7
8      return bless {
9          x => $x,
10         y => $y,
11     }, $class;
12 }
13
14 sub add {
15     my ($self, $other, $swapped) = @_;
16
17     return Point->new(
18         $self->{x} + $other->{x},
19         $self->{y} + $other->{y},
20     );
21 }
22
23 sub to_string {
24     my ($self) = @_;
25
26     return "($self->{x}, $self->{y})";
27 }
```

```
0  my $p1 = Point->new(2, 3);
1  my $p2 = Point->new(4, 5);
2
3  my $p3 = $p1 + $p2;
4
5  print "$p3\n";    # (6, 8)
```

`fallback => 1`: Let Perl derive unspecified operations when reasonable. The method can also be provided by name:

```
0  use overload
1  '+' => 'add',
2  '""' => 'to_string',
3  fallback => 1;
```

An overloaded binary operator normally receives three arguments:

- **\$self**: The overloaded object.
- **\$other**: The other operand.
- **\$swapped**: Whether Perl reversed the operands to find an overloaded method.

The third argument matters for noncommutative operators such as subtraction.

```
0  sub subtract {  
1      my ($self, $other, $swapped) = @_;  
2  
3      if ($swapped) {  
4          return $other - $self->{value};  
5      }  
6  
7      return $self->{value} - $other;  
8  }
```

```
0  $object - 5;  
1  5 - $object;
```

These expressions have different operand orders, so \$swapped helps distinguish them.

- **Common overloadable operations:**

```
0  use overload  
1      '+' => 'add',  
2      '-' => 'subtract',  
3      '*' => 'multiply',  
4      '==' => 'numeric_equal',  
5      'eq' => 'string_equal',  
6      '0+' => 'to_number',  
7      '""' => 'to_string',  
8      'bool' => 'to_boolean',  
9      fallback => 1;
```

- == is numeric comparison.
- eq is string comparison.
- 0+ controls numeric conversion.
- "" controls string conversion.
- bool controls Boolean conversion.

If

```
0  fallback => 0
```

Unspecified operations generally produce an error.

Perl does not normally support C++-style function overloading based on parameter types:

Regular expressions

- **Matching operator:** The most basic regular expression operator is the matching operator, `m/PATTERN/`.
 - Works on `$_` by default.
 - In scalar context, returns true (1) if the match succeeds, or false (the empty string) if the match fails.
 - In list context, returns a list of any parts of the pattern which are enclosed in parentheses. If there are no parentheses, the entire pattern is treated as if it were parenthesised.
 - The `m` is optional if you use slashes as the pattern delimiters.
 - If you use the `m` you can use any delimiter you like instead of the slashes. This is very handy for matching on strings which contain slashes, for instance directory names or URLs.
 - Using the `/i` modifier on the end makes it case insensitive.

```
0  while (<>) {  
1      print if m/foo/;  
2  }
```

- **s/pattern/replacement:** This is the substitution operator, and can be used to find text which matches a pattern and replace it with something else.
 - Works on `$_` by default.
 - In scalar context, returns the number of matches found and replaced.
 - In list context, behaves the same as in scalar context and returns the number of matches found and replaced (a cause of more than one mistake...).
 - You can use any delimiter you want, the same as the `m//` operator.
 - Using `/g` on the end of it matches globally, otherwise matches (and replaces) only the first instance of the pattern.
 - Using the `/i` modifier makes it case insensitive.
- **Binding operators:** If we want to use `m//` or `s///` to operate on something other than `$_` we need to use binding operators to bind the match to another string.
 - `=~` True if the pattern matches
 - `!~` True if the pattern doesn't match

The most common use is to test if a string matches a regular expression. It returns a true value if it matches, and false if it doesn't.

```
0  my $text = "The quick brown fox jumps over the lazy dog";  
1  
2  # Evaluates to true because "fox" is in the string  
3  if ($text =~ /fox/) {  
4      print "Found a fox!\n";  
5  }
```

You can use `=~` with the `s///` operator to find a pattern and replace it. This modifies the original variable on the left

```
0 my $greeting = "Hello World";
1
2 # Replaces "World" with "Perl"
3 $greeting =~ s/World/Perl/;
4
5 print $greeting; # Outputs: Hello Perl
```

When you use matching parentheses `()` in your regular expression, `=~` can be used to extract those specific pieces into variables.

```
0 my $date = "2026-08-20";
1
2 # Captures the digits into $1, $2, and $3
3 if ($date =~ /(\d{4})-(\d{2})-(\d{2})/) {
4     print "Year: $1, Month: $2, Day: $3\n";
5 }
```

does not match a pattern, use the `!~` operator instead.

```
0 my $email = "user-at-domain.com";
1
2 if ($email !~ /@/) {
3     print "Invalid email: Missing '@' symbol.\n";
4 }
```

In Perl, `$1`, `$2`, `$3` (and so on) are special variables that hold captured text from the most recent successful regular expression match.

- **Modifiers:**

- `/i` Make match/substitute match case insensitive
- `/g` Make substitute global (all occurrences are changed)
- `/m` – Multi-line mode. Causes `^` and `$` to match the start and end of any line within the string rather than just the absolute string boundaries.
- `/s` – Single-line mode. Treats the string as a single line, allowing the dot `(.)` metacharacter to match newline `(\n)` characters.
- `/x` – Extended legibility. Tells Perl to ignore unescaped whitespace and allow comments `(#)` inside the regex pattern for clearer layout formatting
- `/e` – Evaluate replacement. Treats the replacement side as a block of Perl code to execute, utilizing its return value as the final replacement text
- `/ee` – Double evaluate. Evaluates the string on the replacement side as a Perl string, and then runs `eval` a second time on the result.
- `/r` – Non-destructive substitution. Instead of modifying the original variable, it performs the operation and returns the mutated string copy.

- /a – ASCII-only restrictions. Forces character classes like \d, \s, and \w to strictly match ASCII characters only.
- /u – Unicode rules. Forces character classes to abide by full Unicode rules (the default setting for most modern Perl scripts).
- /l – Locale rules. Instructs Perl to use the underlying system’s runtime locale setting.
- /d – Default rules. Uses old, backwards-compatible platform defaults.

You can also activate or deactivate modifiers dynamically inside a regex pattern using the (?imsx-imsx) synt

- (?i)apple matches "Apple" or "APPLE" inline.
- (?-i)apple turns off case-insensitivity from that point forward.

- **Meta characters:**

- ^ Start of string
- \$ End of string
- . Any single character except \n
- \n Newline
- \t Matches a tab
- \s Any whitespace character, such as space, tab, or newline
- \S Any non-whitespace character
- \d Any digit (0 to 9)
- \D Any non-digit
- \w Any "word" character - alphanumeric plus underscore (_)
- \W Any non-word character
- \b A word break - the zero-length point between a word character (as defined above) and a non-word character.
- \B A non-word break - anything other than a word break

- **Quantifiers**

- ? 0 or 1
- * 0 or more
- + 1 or more
- {n} match exactly n times
- {n,} match n or more times
- {n,m} match between n and m times

- **Character classes:** Created the usual way with brackets

- ^ negates the class
- - specifies a range of characters. To mach a literal -, it must be either the first or the last character in the class.
- \$, ., (), {}, *, + and other meta characters taken literally

- **Match variable (\$&):** This special variable holds whatever the last regex matched.

- **Alternation:** Alternation is done the usual way

```
0 /^(rabbit|chicken|dog)$/
```

- **Atoms:** In regular expressions, an atom is the most basic, single unit of a pattern match. It either matches a specific set of characters or positions, and quantifiers (like `*` or `+`

We use it to mean "something that is a chunk of regular expression in its own right". Atoms can be arbitrarily created by simply wrapping things in round brackets.

- **Literal characters:** A single ordinary letter or number (like `a` or `7`).
 - **Character classes:** Brackets matching any single character inside them (like `[abc]` or `[\^0-9]`).
 - **The wildcard dot:** The `.` symbol matching any character.
 - **Escaped characters:** Special shorthands (like `\d`, `\w`, or `\s`).
 - **Grouped expressions:** Parentheses surrounding a sub-pattern (like `(abc)`) treated as one combined unit.
- **Capturing matched strings to scalars:** Perl provides an easy way to extract matched sections of a regular expression for later use. Any part of a regular expression that is enclosed in parentheses is captured and stored into special variables. The substring that matches first set of parentheses will be stored in `$1`, and the substring that matches the second set of parentheses will be stored in `$2` and so on. There is no limit on the number of parentheses and associated numbered variables that you can use.

```
0 /(\w)(\w)/; # matches 2 word characters and stores them in
  ↪ $1, $2
1 /(\w+)/; # matches one or more word characters and stores
  ↪ them in $1
```

Parentheses are numbered from left to right by the opening parenthesis.

- **List context:** Evaluating a regular expression in list context is another way to capture information, with parenthesised sub-expressions being returned as a list. We can use this instead of numbered variables if we like:

```
0 $_ = "Our server is training.perltraining.com.au.";
1 my ($full, $host, $domain) = /(([\w-]+)\.([\w.-]+))/;
2
3 print "$1\n"; # prints "training.perltraining.com.au."
4 print "$full\n"; # prints "training.perltraining.com.au."
5 print "$2 : $3\n"; # prints "training : perltraining.com.au."
6 print "$host : $domain\n" # prints "training :
  ↪ perltraining.com.au."
```

Note: A regular expression that fails to match the given string does not always reset `$1`, `$2` etc.

- **Extended regular expressions:** Regular expressions can be difficult to follow at times, especially if they're long or complex. Luckily, Perl gives us a way to split a regular expression across multiple lines, and to embed comments into our regular expression. These are known as extended regular expressions.

- Spaces (including tabs and newlines) in the regular expression are ignored.
- Anything after an un-escaped hash (#) is ignored, up until the end of line.

Extended regular expressions do not alter the format of the second part in a substitution. This must still be written exactly as you wish it to appear.

If you need to include a literal space or hash in an extended expression you can do so by preceding it with a backslash.

By using extended regular expressions, we can change this:

```
0  # Parse a line from 'ls -l'
1  m{^( [\w-]+\s+(\d+)\s+(\w+)\s+(\w+)\s+(\d+)\s+(\w+\s+\d+\s+[\d:]+\s+(.*)$);
```

into

```
0  m{
1      ^                                # Start of line.
2      ([\w-]+\s+                      # $1 - File permissions.
3      (\d+)\s+                        # $2 - Hard links.
4      (\w+)\s+                        # $3 - User
5      (\w+)\s+                        # $4 - Group
6      (\d+)\s+                        # $5 - File size
7      (\w+\s+\d+\s+[\d:]+\s+          # $6 - Date and time.
8      (.*)                            # $7 - Filename.
9      $                               # End of line.
10 }x;
```

- **Greediness:** Regular expressions are, by default, "greedy". This means that any regular expression, for instance `.*`, will try to match the biggest thing it possibly can. Greediness is sometimes referred to as "maximal matching"

```
0  $_ = "The cat sat on the mat";
1
2  /(c.*t)(.*)(m.*t)/;
3
4  print $1; # prints "cat sat on t"
5  print $2; # prints "he "
6  print $3; # prints "mat";
```

It is possible in this example for another set of matches to occur. The first expression `c.*t` could have matched `cat` leaving `sat on the` to be matched by the second expression `.*`. However, to do that, we need to stop `c.*t` from being so greedy.

To make a regular expression quantifier not greedy, follow it with a question mark. For example `.*?`. This is sometimes referred to as "minimal matching".

```
0  $_ = "The fox is in the box.";
1  /(f.*x)/; # greedy -- $1 = "fox is in the box"
2  /(f.*?x)/; # not greedy -- $1 = "fox"
```

- **More meta characters:**

- `\cX` Control character, i.e. CTRL-X
- `\0nn` Octal character represented by nn
- `\xnn` Hexadecimal character represented by nn
- `\l` Lowercase next character
- `\u` Uppercase next character
- `\L` Lowercase until `\E`
- `\U` Uppercase until `\E`
- `\Q` Quote (disable) meta characters until `\E`
- `\E` End of lowercase/uppercase/quote
- `\A` Beginning of string, regardless of whether `/m` is used.
- `\Z` End of string (or before newline at end), regardless of whether `/m` is used.
- `\z` Absolute end of string, regardless of whether `/m` is used.

- **Regexp modifiers for multi-line data:** Perl has two modifiers for multi-line data. `/s` and `/m`. These can be used to treat the string you're matching against as either a single line or as multiple lines. Their presence changes the behaviour of caret (`^`), dollar (`$`) and dot (`.`).

By default caret matches the start of the string. Dollar matches the end of the string (regardless of newlines). Dot matches anything but a newline character.

- Using `/s` forces Perl to treat the target string as a single continuous line, allowing the `.` character to match newlines as well.
- Using `/m` tells Perl that the string contains multiple lines. It changes `^` and `$` to match at the beginning and end of any line within the string (directly after or before an embedded newline).

With the `/s` modifier, caret and dollar behave the same as in the default case, but dot will match the newline character.

With the `/m` modifier, caret matches the start of any line within the string, dollar matches the end of any line within the string. Dot does not match the newline character.

Mode	Specified with	<code>^</code> matches...	<code>\$</code> matches...	Dot matches newline
default	neither <code>/s</code> nor <code>/m</code>	start of string	end of string	No
single-line	<code>/s</code>	start of string	end of string	Yes
multi-line	<code>/m</code>	start of line	end of line	No
clean multi-line	both <code>/m</code> and <code>/s</code>	start of line	end of line	Yes

- **Back references:** There are several special variables related to regular expressions. The parenthesised names beside them are their long names if you use the English module.
 - `$&` is the matched text (MATCH)
 - `$'` (dollar backtick) is the unmatched text to the left of the matched text (PREMATCH)
 - `$'` (dollar forwardtick) is the unmatched text to the right of the matched text (POSTMATCH)
 - `$1`, `$2`, `$3`, etc. The text matched by the 1st, 2nd, 3rd, etc sets of parentheses.

All these variables are modified when a match occurs, and can be used in the same way that other scalar variables can be used.

When Perl sees you using `PREMATCH`, `MATCH`, or `POSTMATCH`, it assumes that you may want to use them again. This means that it has to prepare these variables after every successful pattern match. This can slow a program down because these variables are "prepared" by copying the string you matched against to an internal location.

- **Non-capturing parenthesis:** If you want to use parentheses simply for grouping, and don't want them to set a `$1` style variable, you can use a special kind of non-capturing parentheses, which look like `(?: ... )`

13.1 Advanced Regex

- Subroutine call to a numbered capture group (expression recursion):

```
0  (?i)
```

Re-executes the definition of group *i* to match a new piece of text

- **(?1) or (?g1)**: Calls the first capture group by its absolute index.
- **(?+1) or (?-1)**: Calls a group relative to the current position (e.g., (?-1) calls the previous group).
- **(?&name)**: Calls a named capture group defined by (?<name>...).
- **(?R) or (?0)**: Calls the entire regular expression from the very beginning (global recursion).

```
0  # Matches: Day-Month-Year where each is a 2-digit number
1  # Instead of writing \d{2} three times, we reuse group 1
2  /\b(\d{2})-(?1)-(?1)\b/
```

- **The `\G` anchor**: Every string in Perl has an internal position pointer called `pos()`. When you do a `m//g` match, Perl updates this pointer to the end of the match. The next `m//g` starts searching from that position.

`\G` anchors a match to exactly that position. Not "somewhere after it." Exactly at `pos()`.

```
0  my $str = "aaa bbb ccc";
1
2  # first m//g match finds "aaa" at position 0
3  $str =~ m~\G(\w+)~g;
4  say $1;           # aaa
5  say pos($str);    # 3
6
7  # without \G, the next match could skip ahead
8  # WITH \G, it must match right at position 3
9  $str =~ m~\G(\w+)~g;    # fails! position 3 is a space, not
   ↪ \w
```

```
0  my $str = "aaa bbb ccc";
1
2  $str =~ m~\G(\w+)~g;
3  say $1;
4  say pos($str);
5
6  $str =~ m~\G\s(\w+)~g;
7  say $1;           # bbb
8  say pos($str);    # 7
```

- **The GC flag:** There is a subtlety with `m//g` that trips people up. Normally, when a `m//g` match fails, Perl resets `pos()` to 0. Your position is lost. The next match starts over from the beginning.

The `c` flag changes this. With `m//gc`, a failed match leaves `pos()` where it was. The position survives failure.

- **The `e` modifier:** The replacement side of `s///` is normally a string. Put `/e` on the end, and it becomes Perl code.

```
0 my $text = "I have 3 cats and 12 dogs";
1
2 $text =~ s~(\d+)~$1 * 2~ge;
3
4 say $text;
5 # I have 6 cats and 24 dogs
```

The `/e` modifier tells Perl: "Do not treat the replacement as a double-quoted string. Treat it as code. Evaluate it. Use the result." Since the replacement is code, you can call any function:

- **Embedded code execution inside regex:** That `(?{ })` construct is not a lookahead. It is not a comment. It is live Perl code executing inside the regex engine while the match is still in progress. You have access to the match state, you can modify variables, and you can make decisions based on what has been matched so far.

```
0 use feature 'say';
1
2 "Hello World" =~ m~Hello (?{ say "matched Hello,
   ↪ continuing..." })\w+~;
3 # Output: matched Hello, continuing...
```

The code block fires after `Hello` has matched and before `\w+` starts matching. It is a checkpoint. The engine got this far, so your code runs.

- **Double `e`:**

```
0 $text =~ s~(\d+)~'$1 * 1.13'~ee;
```

First eval produces the string `"100 * 1.13"`. Second eval executes that string as Perl code. Result: 113.

- **Dynamic regex:** You have a list of words. You want to match any of them in a string.

```
0 my @words = qw(foo bar baz quux);
1 my $pattern = join '|', map { quotemeta } @words;
2 $text =~ m~$pattern~;
```

- **Regex recursion:** Handle nested parentheses. Suppose you need to match (something (with nested (parens))).

```
0 my $nested = qr~\(\s*(?: [^()]+ | \s(?! ) \s)* \s*\)\s*x;
```

- Loops with regex:

```
0 my $db = {  
1     name => 'Mike',  
2     age  => '45'  
3 };  
4  
5 qq|@{[ reverse sort keys %{$db} ]}|  
6 =~ s~\S+~print qq|$&: $db->{$&}\n|~ger;
```

References and complex data structures

- **Intro:** There are three main uses for Perl references
 1. **Creating complex data structures:** Perl references can be used to create complex data structures, for instance hashes of arrays, arrays of hashes, hashes of hashes, and more
 2. **Passing arrays and hashes to subroutines and functions:** Since all arguments to subroutines are flattened to a list of scalars, it is not possible to use two arrays as arguments and have them retain their individual identities without using references.
 3. **Object-oriented Perl:** References are used extensively in object oriented Perl. In fact, Perl objects are references to data structures.
- **Creating and dereferencing references:** To create a reference to a scalar, array or hash, we prefix its name with a backslash:

```
0 my $scalar = "This is a scalar";
1 my @array = qw(a b c);
2 my %hash = (
3     'sky' => 'blue',
4     'apple' => 'red',
5     'grass' => 'green',
6 );
7
8 my $scalar_ref = \$scalar;
9 my $array_ref = \@array;
10 my $hash_ref = \%hash;
```

Note that all references are scalars, because they contain a single item of information: the memory address of the actual data

- **ref():** You can find out whether a scalar is a reference or not by using the `ref()` function, which returns a string indicating the type of reference, or `undef` if the scalar is not a reference.
- **Dereferencing:** Dereferencing (getting at the actual data that a reference points to) is achieved by prepending the appropriate sigil to the name of the reference. For instance, if we have a hash reference `$hash_reference` we can dereference it by adding a percentage sign: `[%$hash_reference]`.
- **Arrow notation:** Uses the C arrow concept for a dereferenced access.

```
0 print $$array_ref[0];
1 print $array_ref->[0];
2 print $hash_ref->{"sky"};
```

- **Anonymous data structures:** We can use anonymous data structures to create complex data structures without having to declare many temporary variables. Anonymous arrays are created by using square brackets instead of round ones. Anonymous hashes use curly braces instead of round ones

```

0 my $array_ref = ['a', 'b', 'c', 'd'];
1 my $array_ref = [qw(a b c d)];
2
3 my %transport = (
4     'cars' => [qw(toyota ford holden porsche)],
5     'planes' => [qw(boeing harrier)],
6     'boats' => [qw(clipper skiff dinghy)],
7 );
8
9 my $hash_ref = {
10     a => 1,
11     b => 2,
12     c => 3
13 };

```

Data is pulled out of an anonymous data structure using the arrow notation:

- **Complex data structures:** References are most often used to create complex data structures. Since references are scalars, they can be used as values in both hashes and arrays. This makes it possible to create both deep and complex multi-dimensional data structures.

The use of references in data structures allows you to create arrays of arrays, arrays of hashes, hashes of arrays and hashes of hashes

- **Disambiguation and curly braces:** Consider

```

0 my @res = @$array[0];

```

Does this mean:

- Find @array.
- Look up index 0: \$array[0]
- Turn that (\$array[0]) into an array: @\$array[0]

Or:

- Find the array reference \$array
- Treat that as an array: @\$array
- Take an array slice with index 0: @\$array[0]

Perl does the latter, however if that is what we wanted then we should have written \$\$array[0], as that explicitly returns a single (scalar) result.

We can force Perl to evaluate our expression as the first interpretation above by using curly braces. This allows us to clearly write:

```

0 my @result = @{$array[0]};

```

We can use `${...}`, `@{...}` or `%{...}` syntax to evaluate any expression and dereference the result.

- **Data::Dumper:** Often data structures are very complex, and negotiating and printing these structures can be a tiresome exercise. It's also an unnecessary one, as all the hard work has already been done for you. To save you from having to write specialised printing code in every program for debugging purposes, there's a special library you may find useful called Data::Dumper.

Data::Dumper provides a function which takes Perl data structures and turns them into human readable strings representing the data with in them. It can be used just like this:

```
0 use Data::Dumper;
```

```
0 my %HoH = (  
1     Jacinta => {  
2         age => 26,  
3         favourite_colour => "blue",  
4         sport => "swimming",  
5         language => "Perl",  
6     },  
7  
8     Paul => {  
9         age => 27,  
10        favourite_colour => "green",  
11        sport => "cycling",  
12        language => "Perl",  
13    },  
14 );  
15 print Dumper \%HoH;
```

This will print out something similar to:

```
0 $VAR1 = {  
1     'Paul' => {  
2         'language' => 'Perl',  
3         'favourite_colour' => 'green',  
4         'sport' => 'cycling',  
5         'age' => 27  
6     },  
7  
8     'Jacinta' => {  
9         'language' => 'Perl',  
10        'favourite_colour' => 'blue',  
11        'sport' => 'swimming',  
12        'age' => 26  
13    }  
14 };
```

Not only is this easy to read, but it's also perfectly valid Perl code. This means you can use Data::Dumper to easily give you a structure that you can paste into another program, or which can be 'serialised' to a file and re-created at a later date. Data::Dumper has a lot more uses beyond simple debugging.

Dumper expects to be given one or more references to data structures to dump. If Dumper is provided with a hash or array then every element of the array, or every key and value of the hash, will be considered a separate data structure, and dump separately. The results are not particularly useful:

```
0  # result of: print Dumper %HoH;
1  $VAR1 = 'Paul';
2  $VAR2 = {
3      'language' => 'Perl',
4      'favourite_colour' => 'green',
5      'sport' => 'cycling',
6      'age' => 27
7  };
8  $VAR3 = 'Jacinta';
9  $VAR4 = {
10     'language' => 'Perl',
11     'favourite_colour' => 'blue',
12     'sport' => 'swimming',
13     'age' => 26
14  };
```

- **Hard vs soft references:** Hard references point directly to underlying data values in memory, while symbolic references store the string name of a variable and look it up in the symbol table.

Hard references are created using the backslash operator or by declaring anonymous data structures

```
0  my $scalar_ref = \$name;           # Reference to a scalar
1  my $array_ref  = \@colors;         # Reference to an array
2  my $hash_ref   = \%user_data;      # Reference to a hash
3  my $sub_ref    = \&my_function;    # Reference to a subroutine
```

```
0  my $array_ref = [ 'apple', 'banana', 'cherry' ];
1  my $hash_ref = { name => 'Alice', age => 30 };
```

the `\\;( ... )` syntax distributes a backslash to everything inside the parentheses.

Soft references are not distinct data types; they are just regular string scalars containing the name of a variable. They are created simply by assigning a string and then dereferencing it.

```
0  no strict 'refs'; # Required to allow symbolic references
```

You assign a variable's name (as a string) to a scalar, and then dereference that scalar using standard type prefixes

```
0 no strict 'refs'; # Required to allow symbolic references
1
2 our $animal = "Cat"; # Must be a package/global variable, not
   ↳ 'my'
3 my $pointer = "animal";
4
5 print $$pointer; # Outputs: Cat (Looks up the variable named
   ↳ "animal")
```

```
0 no strict 'refs';
1
2 our @fruits = ('apple', 'orange');
3 my $list_name = "fruits";
4
5 print $list_name->[0]; # Outputs: apple
```

External Files and Packages

- **Require:** Perl implements a number of mechanisms for loading code from external files. The most simplest of these is by using the require function:

```
0  require 'file.pl';
```

Perl is smart enough to make sure that the same file will not be included twice if it's required through the same specified name.

```
0  # The file is only included once in the following case:
1  require 'file.pl';
2  require 'file.pl';
```

Required files must end with a true value. This is usually achieved by having the final statement of the file being:

```
0  1;
```

Conflicts can occur if our included file declares subroutines with the same name as those that appear in our main program. In most circumstances the subroutine from the included file takes precedence, and a warning is given.

The use of require has been largely deprecated by the introduction of modules and the use keyword. If you're writing a code library from scratch we recommend that you create it as a module. However, require is often found in legacy code and is a useful thing to understand.

Any code in the file (except for subroutines) will be executed immediately when the file is required. The require occurs at run-time, this means that Perl will not throw an error due to a missing file until that statement is reached, and any subroutines inside the file will not be accessible until after the require.

Variables declared with my are not shared between files, they are only visible inside the block or file where the declaration occurs. To share packages between files we use package variables which are covered later in this chapter.

The use of modules (which we will learn about later) allows for external files to be loaded at compile-time, rather than run-time.

- **Use strict and warnings:** Perl pragmas, such as strict and warnings are lexically scoped. Just like variables declared with my, they last until the end of the enclosing block, file or eval.

This means that you can turn strict and warnings on in one file without it influencing other parts of your program. Thus, if you're dealing with legacy code, then your new libraries, modules and classes can be strict and warnings compliant even though the older code is not.

- **Require example:**

```

0  # Greetings.pl
1  use strict;
2  use warnings;
3  my %greeting_in = (
4      en => "Hello",
5      'en-au' => "G'day",
6      fr => "Bonjour",
7      jp => "Konnichiwa",
8      zh => "Nihao",
9  );
10
11 sub hello {
12     my $language = shift "en";
13
14     my $greeting = $greeting_in{$language}
15         or die "Don't know how to greet in $language";
16
17     return $greeting;
18 }
19
20 1;

```

```

0  # program.pl
1  use strict;
2  require "Greetings.pl";
3
4  print "English: ", hello("en"), "\n"; # Prints "Hello"
5  print "Australian: ", hello("en-au"), "\n"; # Prints "G'day"

```

- **Packages:** When working with a large project, the chances of naming conflicts increases. Two entirely different files may have two different subroutines with the same name; however it is only the last one loaded that will be used by Perl. Files from different projects may be re-used in new developments, and these may have considerable name clashes. Multiple files can also make it difficult to determine where subroutines are originally declared, which can make debugging difficult.

Perl's packages are designed to overcome these problems. Rather than just putting code into separate files, code can be placed into independent packages, each with its own namespace. By ensuring that package names remain unique, we also ensure that all subroutines and variables can remain unique and easily identifiable.

A single file can contain multiple packages, but convention dictates that each file contains a package of the same name. This makes it easy to quickly locate the code in any given package.

Writing a package in Perl is easy. We simply use the package keyword to change our current package. Any code executed from that point until the end of the current file or block is done so in the context of the new package.

```

0  package Greetings;
1  use strict;
2  use warnings;
3
4  my %greeting_in = (
5      en => "Hello",
6      'en-au' => "G'day",
7      fr => "Bonjour",
8      jp => "Konnichiwa",
9      zh => "Nihao",
10 );
11
12 sub hello {
13     my $language = shift "en";
14     my $greeting = $greeting_in{$language}
15         or die "Don't know how to greet in $language";
16
17     return $greeting;
18 }
19
20 1;

```

The package that you're in when the Perl interpreter starts (before you specify any package) is called `main`. Package declarations use the same rules as `my`, that is, it lasts until the end of the enclosing block, file, or `eval`.

Perl convention states that package names (or each part of a package name, if it contains many parts) starts with a capital letter. Packages starting with lower-case are reserved for pragmas (such as `strict`).

- **Scoping operator:** Perl provides a scoping operator in the form of a pair of adjacent colons. The scoping operator allows us to refer to information inside other packages, and is usually pronounced "double-colon".

```

0  require "Greetings.pl";
1
2  print Greetings::hello("en"), "\n";
3  print Greetings::hello("jp"), "\n";
4
5  # This calls the hello() subroutine in our main package
6  # (below), printing "Greetings Earthling".
7  print hello(), "\n";
8
9  sub hello {
10     return "Greetings Earthling";
11 }

```

- **Changing variables in other packages:** Occasionally we may wish to change the value of a variable in another package. It should be very rare that we should need to do this, and it's not recommended you do so unless this is a documented feature of your package. However, in the case where we do need to do this, we use the scoping operator again.

```

0  use Carp;
1
2  # Turning on $Carp::Verbose makes carp() and croak() provide
3  # stack traces, making them identical to cluck() and
   ↪ confess().
4  # This is documented in 'perldoc Carp'.
5
6  $Carp::Verbose = 1;

```

`carp()` and `croak()` are functions from the Carp module that report errors from the calling code's perspective instead of inside your module

- **`carp()`**: Works like `warn()`, sending a non-fatal warning, but attributes the line number to the code that called your module.
- **`croak()`**: Works like `die()`, throwing a fatal error, but attributes the line number to the code that called your module rather than the exact line inside the function.

`cluck()` and `confess()` are error-handling functions provided by the Carp module that include a full stack backtrace to show exactly where and how an error occurred

- `cluck()` acts like an enhanced `warn()`. It prints the error message and the full stack trace, but allows the program to keep running.
- `confess()` acts like an enhanced `die()`. It prints the error message and the full stack trace, then terminates the program immediately.

```

0  use Carp qw(cluck confess); # Explicitly importing cluck

```

- **Main package**: There's a shorthand for accessing variables and subroutines in the main package, which is to use double-colon without a package name. This means that `$::foo` is the same as `$main::foo`
- **Package variables and our**: It is not possible to access lexically scoped variables (those created with `my`) outside of their enclosing block. This means that we need another way to create variables to make them globally accessible. These global variables are called package variables, and as their name suggests they live inside their current package. The preferred way to create package variables, under Perl 5.6.0 and above, is to declare them with the `our` statement. Of course, there are alternatives you can use with older version of Perl, which we also show here:

```

0  package Carp;
1  our $VERSION = '1.01'; # Preferred for Perl 5.6.0 and above
2
3  # use vars qw/$VERSION/; # Preferred for older versions
4  # $VERSION = '1.01';
5
6  # $Carp::VERSION = '1.01'; # Acceptable but requires that we
   ↪ then
7  # always use this full name under strict

```

In all of the cases above, both our package and external code can access the variable using `$Carp::VERSION`.

Modules

- **Module:** A module is a separate file containing Perl source code, which is loaded and executed at compile time. This means that when you write:

```
0  use CGI;
```

Perl looks for a file called CGI.pm (.pm for Perl Module), and upon finding it, loads it in and executes the code inside it, before looking at the rest of your program.

Sometimes you might choose to pass extra information to the module when you load it. Often this is to request the module create new subroutines in your namespace.

```
0  use CGI qw(:standard);
1  use File::Copy qw(copy);
```

Note the use of qw(), this is a list of words (in our case, just a single word). It's possible to pass many options to a module when you load it. In the case above, we're asking the CGI module for the :standard bundle of functions, and the File::Copy module for just the copy subroutine.

Each module has a different set of options (if any) that it will accept. You need to check the documentation of the module you're dealing with to which (if any) are applicable to your needs.

- **Require vs use:** The fundamental difference is that use happens at compile time, while require happens at runtime

The use statement is actually a built-in shortcut. Writing use MyModule; is exactly equivalent to executing

```
0  BEGIN {
1      require MyModule;
2      MyModule->import();
3  }
```

Feature	use	require
Execution Time	Compile time (processed before the rest of the script runs)	Runtime (processed only when the execution line is hit)
Implicit Import	Yes. Automatically calls the module's import method.	No. Does not call import automatically.
File Types	Only loads Perl Modules (.pm files).	Loads both libraries (.pl) and modules (.pm).
Syntax	Expects a bareword name without an extension (e.g., use File::Copy;).	Accepts a bareword or a string expression with a file extension.
Conditionals	Cannot be placed inside if blocks (it will still load at compile time).	Can be placed inside if statements for conditional loading.

- **No import method:** If you do not write an import method and do not inherit one (like from `Exporter`), your module will still load perfectly fine, but it will not export anything automatically into the caller's namespace.

Perl does not require an import method. If you use `use MyModule;` and `MyModule` has no import subroutine, Perl simply skips the step without throwing an error or warning.

Because no functions or variables are copied into the main script's namespace, you cannot call them by their short names. You must use their fully qualified package paths.

```
0  # --- MyModule.pm ---
1  package MyModule;
2  sub say_hello {
3      print "Hello from the module!\n";
4  }
5  1;
6
7  # --- main.pl ---
8  use MyModule;
9
10 # say_hello();          # This will crash! Perl doesn't know
    ↳ what say_hello is.
11 MyModule::say_hello(); # This works perfectly.
```

- **Import method:** Because writing a manual import method to manipulate symbol tables is complex, almost all Perl developers inherit the behavior from the core `Exporter` module.

```

0  package MyMath;
1  use strict;
2  use warnings;
3
4  # Import the standard 'import' method from the Exporter
   ↳ module
5  use Exporter qw(import);
6
7  # Functions in @EXPORT are automatically imported by default
8  our @EXPORT = qw(add);
9
10 # Functions in @EXPORT_OK are only imported if explicitly
   ↳ requested
11 our @EXPORT_OK = qw(multiply);
12
13 sub add {
14     my ($a, $b) = @_;
15     return $a + $b;
16 }
17
18 sub multiply {
19     my ($a, $b) = @_;
20     return $a * $b;
21 }
22
23 1; # Modules must return a true value

```

- **%EXPORT_TAGS**: Group related exportable symbols into convenient shortcut tags. This allows developers importing your module to request an entire category of functions or variables at once by using a colon prefix (:tag_name)

Every symbol listed inside

You cannot nest tag names inside @EXPORT or @EXPORT_OK

Define the groups using an array reference for each tag name.

```

0  our %EXPORT_TAGS = (
1      utils => [ qw( validate_email ) ],
2      user  => [ qw( create_user delete_user ) ],
3      log   => [ qw( log_info log_error ) ],
4      all   => [ @EXPORT, @EXPORT_OK ], # Quick shortcut
   ↳ combining everything
5  );

```

When loading the module, prepend a colon (:) to the tag name to pull in that entire group

```

0  # Import only the user management group (:user)
1  use MyModule qw(:user);
2
3  create_user(); # Works!
4  delete_user(); # Works!
5
6  # log_info(); # Fails! (Not imported)

```

- **Advanced import techniques:** You can combine tags, default exports, and exclusions using specific prefixes
 - **:DEFAULT:** Refers implicitly to everything inside your module's @EXPORT array.
 - **!:** Negates/excludes a specific symbol from being imported.

```

0  # Imports default functions, the logging tag, but EXCLUDES
   ↪ log_error
1  use MyModule qw(:DEFAULT :log !log_error);

```

- **Where does Perl look for modules?:** Perl searches through a list of directories that are determined when the Perl interpreter is compiled. You can see this list (and all the other options Perl was compiled with), by using perl -V

The list of directories which Perl searches for modules is stored in the special variable @INC. It's possible to change @INC so that Perl will search in other directories as well. This is important if you have installed your own private copy of some modules.

Of course, being Perl, there's more than one way to change @INC. Here are some of the ways to add to the list of directories inside @INC:

- Call Perl with the -I command-line switch with the location of the extra directory to search. For example:

```

0  perl -I/path/to/libs

```

This can be done either in the shebang line, or on the command-line.

- Use the lib pragma in your script to inform Perl of extra directories. For example:

```

0  use lib "/path/to/libs";

```

- Setting the PERL5LIB environment variable with a colon-separated list of directories to search. Note that if your script is running with taint checks this environment variable is ignored.

Since use statements occur before regular Perl code is executed, modifying @INC directly usually does not have the desired effect.

- **Finding installed modules:** Perl comes with many modules in its standard distribution. You can get a list of all of them by doing a perldoc perlmodlib.

Besides from the modules in the standard distribution, you can also see any other modules that were installed on your system by using `perldoc perllocal`. Generally this file only lists other modules that were installed by hand, or using one of the CPAN installers (more on this later). Modules installed through your operating system's packaging system may not appear in `perldoc perllocal`.

To find a complete list of modules available on your system, regardless of how they were installed, read the documentation provided by `perldoc -q` installed.

You can get more information on any module that you have installed by using `perldoc module_name`. For example, `perldoc English` will give you information about the `English` module. You can also use `perldoc -l module_name` to locate a particular module, and `perldoc -m module_name` to view the source of a module.

- **Using CPAN (Comprehensive Perl Achieve Network) modules:** CPAN provides more than 9,000 separate and freely available modules. This makes CPAN an excellent starting point when you wish to find modules to help solve your particular problem. However, you should keep in mind that not all CPAN modules are created equal. Some are much better documented and written than others. Some (such as the `CGI` or `DBI`) modules have become de-facto standards, whereas others may not be used by anyone except the module's author.

As with any situation when you're using third party code, you should take the time to determine the suitability of any given module for the task at hand. However, in almost all circumstances it's better to use or extend a suitable module from CPAN rather than trying to re-invent the wheel.

Many of the popular CPAN modules are pre-packaged for popular operating systems. In addition, the CPAN module that comes with Perl can make the task of finding and installing modules from CPAN much easier.

Most CPAN modules come with `README` and/or `INSTALL` files which tell you how to install the modules. This may vary between operating systems. On Unix and Unix-like operating systems the process is usually:

```
0  perl Makefile.PL
1  make
2  make test
3  make install
```

For ActiveState Perl installations (which includes most Microsoft Windows machines) the use of PPM (Programmer's Package Manager) is recommended. PPM provides a command line interface for downloading and installing pre-compiled versions of most CPAN modules.

- **Writing modules:** Modules contain regular Perl code, and for most modules the vast majority of that code is in subroutines. Sometimes there are a few statements which initialise variables and other things before any of those subroutines are called, and those get executed immediately. The subroutines get compiled and tucked away for later use.

Besides from the code that's loaded and executed, two more special things happen. Firstly, if the last statement in the module did not evaluate to true, the Perl compiler throws an exception (usually halting your program before it even starts). This is so that a module could indicate that something went wrong, although in reality this feature is almost never used. Virtually any Perl module you care to look at will end with the statement `1;` to indicate successful loading.

The other thing that happens when a module is used is that its import subroutine (if one exists) gets called with any directives that were specified on the `use` line. This is useful if you want to export functions or variables to the program that's using your module for functional programming but is almost never used (and very often discouraged) for object oriented programming.

As you've no doubt guessed by now, modules and packages often go hand-in-hand. We know how to use a module, but what are the rules on writing one? The big one is this: A module is a file that contains a package of the same name.

That's it. So if you have a package called `Tree::Fruit::Citrus::Lime`, the file would be called `Tree/Fruit/Citrus/Lime.pm`, and you would use it with `use Tree::Fruit::Citrus::Lime;`

A module can contain multiple packages if you desire. So even though the module is called `Chess::Piece`, it might also contain packages for `Chess::Piece::Knight` and `Chess::Piece::Bishop`. It's usually preferable for each package to have its own module, otherwise it can be confusing to your users how they can load a particular package.

When writing modules, it's important to make sure that they are well-named, and even more importantly that they won't clash with any current or future modules, particularly those available via CPAN. If you are writing a module for internal use only, you can start its name with `Local::` which is reserved for the purpose of avoiding module name clashes.

Note: A module is a file that contains a package of the same name.

- **Use vs require:** Perl offers several different ways to include code from one file into another. `use` is built on top of `require` and has the following differences:
 - Files which are used are loaded and executed at compile-time, not run-time. This means that all subroutines, variables, and other structures will exist before your main code executes. It also means that you will immediately know about any files that Perl could not load.
 - `use` allows for the import of variables and subroutines from the used package into the current one. This can make programming easier and more concise.
 - Files called with `use` can take arguments. These arguments can be used to request special features that may be provided by some modules.

Both methods:

- Check for redundant loading, and will skip already loaded files.
- Raise an exception on failure to find, compile or execute the file.
- Translate `::` into your systems directory separator.

Where possible `use` and Perl modules are preferred over `require`.

- **extern equivalent:** The equivalent of an `extern` global variable from C in Perl is an **our** variable declared in a shared package or module.

```

0  package MyConfig;
1  use strict;
2  use warnings;
3
4  # Declare a global variable
5  our $GlobalSetting = "Active";
6
7  1;

```

```

0  use strict;
1  use warnings;
2  use MyConfig;
3
4  # Access the extern-like global variable using its full
   ↪ package path
5  print $MyConfig::GlobalSetting;
6
7  # Or bring it into your current namespace
8  our $GlobalSetting;
9  print $GlobalSetting;

```

- **Static equivalent:** A my variable is exactly how you restrict scope in Perl, but it behaves like a C static in two distinct ways depending on where you declare it.

If you declare a my variable at the top level of a file, it is invisible to other files. This behaves exactly like a file-level static variable in C.

```

0  # Inside MyModule.pm
1  use strict;
2
3  my $file_private = 42; # Visible only inside this file
4
5  sub get_value { return $file_private; }

```

If you want a variable inside a function to keep its value between calls (like a function-level static in C), use Perl's state keyword instead of my. A my variable inside a function is reinitialized every time the function runs.

- **Warnings and strict:** When your module is used by a script, whether or not it runs with warnings depends upon whether the calling script is running with warnings turned on. You can (and should) invoke the use warnings pragma to turn on warnings for your module without changing warnings for the calling script.

Your modules should always use strict.

```

0  use strict;
1  use warnings;

```

- **Exporting and importing subroutines:** Writing your own import function for each and every module would be a tiresome and error-prone process. However, Perl comes with a module called `Exporter`, which provides a highly flexible interface with optimisations for the common case.

`Exporter` works by checking inside your module for three special data structures, which describe both what you wish to export, and how you wish to export them. These structures are

- `@EXPORT` symbols to be exported into the user's name space by default
 - `@EXPORT_OK` symbols which the user can request to be exported
 - `%EXPORT_TAGS` that allows for bundles of symbols to be exported when the user requests a special export tag.
- **@ISA:** To take advantage of `Exporter`'s import function we need to let Perl know that our package has a special relationship with the `Exporter` package. We do this by telling Perl that we inherit from `Exporter`. Our package and the rest of our program does not need to be written in an object oriented style for this to work.

Now when Perl goes looking for the import function it will first look in our package. If it can't be found there, Perl will look for a special array called `@ISA`. The contents of the `@ISA` array is interpreted as a list of parent classes, and each of these will be searched for the missing method.

To specify that this package is a sub-class of the `Exporter` module we include the following lines:

```
0  use Exporter;
1  our @ISA = qw(Exporter);
```

Note: `ISA` means is-a.

- **use base:** An alternative to adding parent modules to `@ISA` yourself is to use the `base` pragma. This allows you to declare a derived class based upon the listed parent classes. Thus the two lines above becomes:

```
0  use base qw(Exporter);
```

Here's an example of just using `@EXPORT` and `@EXPORT_OK`. Our hypothetical module, `People::Manage` is used for managing interpersonal relations.

```
0  package People::Manage;
1  use base qw(Exporter);
2  use vars qw(@EXPORT @EXPORT_OK);
3
4  @EXPORT = qw(invite $name @friends %addresses); # invite is a
   ↳ subroutine
5  @EXPORT_OK = qw(&taunt $spouse @enemies %postcodes); # so is
   ↳ taunt
```

The base pragma takes care of ensuring that the Exporter module is loaded.

The base pragma is available for all versions of Perl above 5.6.0.

- **Polluting the caller's namespace:** Exporting your symbols by default, by populating the `@EXPORT` array, means that anyone using your module will receive these symbols without having to ask for them. This is generally considered to be bad style, and is sometimes referred to as 'polluting' the caller's namespace.

Using `@EXPORT_OK` allows the user to choose which symbols they wish to bring into their name space. All other symbols can be accessed by using their full name, such as `People::Manage::invite()`, when required.

- **Importing symbols:** Once your module is written and it exports a few symbols, it's time to use it. This is done with the `use` command that we've seen with `strict` and other modules. We can load our module in three ways:
 1. use `People::Manage`; which imports all of the symbols stored in `@People::Manage::EXPORT`.
 2. use `People::Manage ()`; which imports none of the symbols in either `@People::Manage::EXPORT` or `@People::Manage::EXPORT_OK`.
 3. use `People::Manage qw($name $spouse invite)`; which imports all the listed symbols. If a symbol is mentioned which is not in either `@People::Manage::EXPORT` or `@People::Manage::EXPORT_OK` then a fatal error will occur.
- **FindBin:** Import the `$RealBin` variable directly into your script's namespace, which holds the absolute, canonical path to the directory containing your script.

```
o use FindBin qw($RealBin);
```

Unlike the default `$Bin` variable, `$RealBin` resolves all symbolic links (symlinks). This means if your script is a shortcut pointing somewhere else, `$RealBin` tells you where the physical file actually lives rather than where the shortcut is located.

Then, we can do something like

```
o use lib ($RealBin, "$RealBin/lib");
```

Using Perl Objects (Perl OOP)

- **OOP in perl:** In Perl, object-oriented programming (OOP) is built on a few deceptively simple rules rather than dedicated language keywords. Classic Perl OOP relies on three core principles: a class is simply a package, an object is a reference that "knows" its class, and a method is a standard subroutine

In Perl, there is no class keyword (in older/classic versions). Instead, a package acts as the class. The package defines a separate namespace to prevent variable and subroutine names from colliding with other parts of the program

reference to a hash that has been explicitly linked to a specific class. This linking is achieved using the built-in **bless** function.

By convention, the constructor method is named **new**, though you can technically name it anything

```
0  package Animal;
1  use strict;
2  use warnings;
3
4  # The Constructor
5  sub new {
6      my $class = shift;           # Captures the class name
7      ↪ ("Animal")
8      my $self = {                 # Creates a reference to an
9      ↪ empty hash
10         name => shift,
11         type => shift,
12     };
13
14     # "Blesses" the hash reference into the class namespace
15     return bless $self, $class;
16 }
17
18 1; # Packages must return a true value
```

The **bless** function turns a standard reference into an object by associating it with a specific package (class).

Unlike languages with strict object-oriented syntax, an object in Perl is simply a regular data structure (usually a hash reference) that has been explicitly told which class it belongs to. Once a reference is "blessed", you can call the package's subroutines as object methods

two-argument form is highly recommended to support class inheritance safely:

```
0  bless REF, CLASSNAME;
1  bless REF;
```

- **REF:** Any valid Perl reference (Hash, Array, Scalar, Subroutine, etc.). Hash references are the most common because they allow you to store named attributes.
- **CLASSNAME:** A string containing the name of the package/class. If omitted, Perl blesses the reference into the current package.
- **Return Value:** It returns the newly blessed reference, allowing you to use it instantly or return it at the end of a constructor

Then, “When this reference is used as an object, look for its methods in this package.”

```

0  package Person;
1
2  sub new {
3      my ($class, $name, $age) = @_;
4
5      my $self = {
6          name => $name,
7          age  => $age,
8      };
9
10     bless $self, $class;
11
12     return $self;
13 }

```

- **Methods and the arrow syntax:** Methods are just subroutines defined inside the package. When you invoke a method using the arrow operator (`->`), Perl automatically passes the object itself (or the package name, if it's a class method) as the first hidden argument
 - **Defining a Method:** You must always shift the object (`$self`) off the argument array (`@_`) first.
 - **Getters and Setters:** Because attributes are just keys inside a blessed hash, you write subroutines to read or modify them

```

0  sub speak {
1      my $self = shift; # The object itself is passed first
2      print $self->{name} . " says hello!\n";
3  }

```

```

0  use Animal;
1
2  # Create the object (instantiation)
3  my $dog = Animal->new("Fido", "Dog");
4
5  # Call a method
6  $dog->speak(); # Outputs: Fido says hello!

```

- **Inheritance:** Inheritance is handled by manipulating a special package array called `@ISA` (which stands for “is a”). Modern classic Perl uses the `parent` directive to manage this cleanly

```

0  package Dog;
1  use parent 'Animal'; # Inherits all methods from Animal
2  1;

```

- **Tied variables:** In Perl, a tied variable looks like an ordinary scalar, array, hash, or filehandle, but operations on it are handled by methods in a Perl class.

This lets you change what assignment, retrieval, deletion, and other normal variable operations actually do.

For example, a tied hash could store its values in a database or file instead of ordinary memory:

You connect a variable to a class using tie:

```

0  tie VARIABLE, CLASS, ARGUMENTS;

```

```

0  tie %hash, "MyHash", "data.txt";

```

Perl creates an object from MyHash and forwards operations on %hash to that object.

The methods your class needs depend on the variable type.

– **Scalar:**

```

0  TIESCALAR
1  FETCH
2  STORE

```

– **Array:**

```

0  TIEARRAY
1  FETCH
2  STORE
3  FETCHSIZE
4  STORESIZE
5  CLEAR
6  PUSH
7  POP
8  SHIFT
9  UNSHIFT
10 SPLICE
11 EXISTS
12 DELETE

```

Not every method is always required, but unsupported operations will fail or behave incorrectly.

– **Hash:**

```
0  TIEHASH
1  FETCH
2  STORE
3  DELETE
4  CLEAR
5  EXISTS
6  FIRSTKEY
7  NEXTKEY
8  SCALAR
```

FIRSTKEY and NEXTKEY support iteration through operations such as:

```
0  keys %hash;
1  each %hash;
```

– **Filehandle:**

```
0  TIEHANDLE
1  PRINT
2  PRINTF
3  READ
4  READLINE
5  WRITE
6  CLOSE
```

```

0  package LoggedScalar;
1
2  sub TIESCALAR {
3      my ($class, $initial_value) = @_;
4
5      return bless {
6          value => $initial_value
7      }, $class;
8  }
9
10 sub FETCH {
11     my ($self) = @_;
12
13     print "Scalar was read\n";
14     return $self->{value};
15 }
16
17 sub STORE {
18     my ($self, $new_value) = @_;
19
20     print "Scalar was changed to $new_value\n";
21     $self->{value} = $new_value;
22 }

```

```

0  package main;
1
2  tie my $number, "LoggedScalar", 10;
3
4  print "$number\n";    # Calls FETCH
5  $number = 25;        # Calls STORE
6  print "$number\n";    # Calls FETCH

```

- **Tied and untied:** tied returns the underlying object or undef if the variable is not tied.

```

0  my $object = tied($number);
1
2  print $object->{value};

```

untie removes the connection:

```

0  untie $number;

```

Afterward, \$number is once again an ordinary scalar.

A class can define UNTIE to perform cleanup:

```
0  sub UNTIE {  
1      my ($self, $reference_count) = @_;  
2      print "Variable was untied\n";  
3  }
```

It can also define DESTROY, which runs when the underlying object is destroyed.

Threads

- **Threads:** Perl supports threads through the core threads module. However, Perl threads differ from lightweight POSIX threads: each Perl thread receives its own Perl interpreter and initially gets copies of the parent thread's variables.

These are commonly called **interpreter threads**, or **ithreads**.

```
0  use strict;
1  use warnings;
2  use threads;
3
4  sub do_work {
5      my ($name) = @_;
6
7      print "Hello from $name\n";
8      return 42;
9  }
10
11 my $thread = threads->create(\&do_work, "worker");
12
13 my $result = $thread->join();
14
15 print "Worker returned: $result\n";
```

`threads->create()` starts a new thread and returns a thread object. The new thread executes *do_work*.

`join()` waits for the thread to finish and retrieves its return value.

You can also use an anonymous subroutine

```
0  my $thread = threads->create(sub {
1      print "Running in another thread\n";
2      return 100;
3  });
4
5  my $result = $thread->join();
```

Ordinary variables are copied when the thread is created:

```

0  use threads;
1
2  my $number = 10;
3
4  my $thread = threads->create(sub {
5      $number = 50;
6      print "Worker: $number\n";
7  });
8
9  $thread->join();
10
11 print "Main: $number\n";

```

Changing \$number inside the worker does not change the parent thread's copy.

- **Sharing variables:** Use `threads::shared` when multiple threads must access the same variable:

```

0  use strict;
1  use warnings;
2  use threads;
3  use threads::shared;
4
5  my $counter :shared = 0;
6
7  my @workers;
8
9  for (1 .. 4) {
10     push @workers, threads->create(sub {
11         for (1 .. 1_000) {
12             lock($counter);
13             ++$counter;
14         }
15     });
16 }
17
18 $_->join() for @workers;
19
20 print "Counter: $counter\n";

```

- **Locks:** A race condition occurs if multiple threads modify shared state concurrently. `lock()` gives the current thread exclusive access to a shared variable for the remainder of the enclosing scope:

```

0  {
1      lock($counter);
2      ++$counter;
3  }

```

The lock is automatically released when execution leaves the block. When locking multiple variables, always acquire the locks in a consistent order. Otherwise, two threads can wait on one another indefinitely—a condition known as deadlock.

Imagine we have Account A and Account B.

- Thread 1 wants to transfer \$100 from Account A to Account B. It locks Account A first, then tries to lock Account B.
 - Thread 2 wants to transfer \$50 from Account B to Account A. It locks Account B first, then tries to lock Account A.
1. Thread 1 acquires the lock for Account A.
 2. Thread 2 acquires the lock for Account B.
 3. Thread 1 attempts to acquire the lock for Account B and blocks (because Thread 2 holds it).
 4. Thread 2 attempts to acquire the lock for Account A and blocks (because Thread 1 holds it).

Neither thread can proceed, and they will wait on each other indefinitely.

Note: Arrays and hashes can also be shared. For complicated shared structures, you may need `shared_clone()`:

`shared_clone` is a helper function provided by the `threads::shared` module. It is designed to take a reference to a standard, non-shared data structure, perform a deep copy, and return a version of that structure that can safely be shared across different interpreter threads (ithreads).

If you try to assign a normal, unshared reference into a shared array or hash, Perl will throw an error like "Invalid value for shared scalar". `shared_clone` solves this by recursively cloning and sharing every component of the structure.

- **Thread queues:** For producer-consumer programs, `Thread::Queue` is generally cleaner than manually sharing arrays and managing locks:

When writing multi-threaded Perl programs, multiple threads cannot safely read or write to a standard array at the same time without causing data corruption. `Thread::Queue` solves this problem by completely handling the underlying locks and synchronization, allowing any number of threads to push or pull data safely

- **`Thread::Queue->new()`:** Creates a brand new, empty queue.
- **`enqueue(LIST)`:** Adds one or more items to the end of the queue.
- **`dequeue()`:** Removes and returns the item at the front of the queue. Crucially, if the queue is empty, `dequeue()` will pause (block) the execution of that thread until another thread enqueues something.
- **`dequeue_nb()`:** A non-blocking version of `dequeue`. If the queue is empty, it returns `undef` immediately instead of waiting.
- **`pending()`:** Returns the number of items currently waiting in the queue.
- **`end()`:** Tells the queue that no more items will be added. Any threads blocking on a `dequeue()` will unblock, and once the remaining items are consumed, `dequeue()` will return `undef`.

The most common use case for `Thread::Queue` is a producer-consumer pattern, where a main script splits tasks up and feeds them to worker threads.

```
0  use strict;
1  use warnings;
2  use threads;
3  use Thread::Queue;
4
5  my $task_queue = Thread::Queue->new();
6
7  # Create worker threads
8  my @workers;
9  for (1..3) {
10     push @workers, threads->create(\&worker_logic);
11 }
12
13 # Feed data into the queue (The Producer)
14 for my $job_id (1..10) {
15     print "Main: Enqueuing job $job_id\n";
16     $task_queue->enqueue($job_id);
17 }
18
19 # Signal workers that no more data is coming
20 $task_queue->end();
21
22 # Wait for all threads to finish up
23 $_->join() for @workers;
24 print "Main: All jobs completed!\n";
25
26
27 # --- Worker Subroutine ---
28 sub worker_logic {
29     my $tid = threads->t看id(); # Get current thread ID
30
31     # dequeue() will automatically return undef after
32     ↪ $task_queue->end()
33     # AND the queue becomes completely empty.
34     while (defined(my $job = $task_queue->dequeue())) {
35         print "Worker $tid: Processing job $job\n";
36         sleep(1); # Simulate processing time
37     }
38     print "Worker $tid: Shutting down.\n";
39 }
```

Enqueuing undef causes your program to hang because Perl's `enqueue()` method discards undef values instead of adding them to the queue.

You do not need to write your own locks because `Thread::Queue` automatically wraps all of its internal data manipulations in thread-safe locks using Perl's native `threads::shared` mechanisms. Every time you call a method like `enqueue` or `dequeue`, the module handles the complex locking logic for you.

When the main thread adds an item, the module executes these precise steps:

1. **Acquires an Internal Lock:** It locks the queue's internal storage array so no other thread can read or write to it at that exact microsecond.
2. **Filters the Input:** It checks the data. If it is undef, it drops it. If it is a complex data structure (like a hash reference), it clones it so it can safely exist in shared memory.
3. **Pushes the Data:** It appends the item to the end of the array.
4. **Signals Waiting Threads:** It issues a `cond_signal()` (conditional signal). This acts like an alarm clock telling any sleeping worker threads, "Hey, data just arrived!"
5. **Releases the Lock:** It unlocks the array so other threads can use it.

When a worker thread asks for a job, the module performs this sequence:

1. Acquires an Internal Lock: It locks the internal array to ensure exclusive access.
2. Checks for Data:
 - If data exists: It shifts the first item off the front of the array, releases the lock, and returns the item to the worker.
 - If the queue is empty: It places the worker thread into a specialized sleep state called `cond_wait()`. Crucially, entering this sleep state temporarily releases the lock so other threads aren't blocked from adding data.
3. **Wakes Up:** When an `enqueue()` triggers the `cond_signal()`, the sleeping worker wakes up, re-acquires the lock, grabs the newly arrived item, releases the lock, and goes to work.

In multi-threaded programming, you can queue almost anything, but the golden rule is to queue the instructions or identifiers for the work, rather than the massive data objects themselves.

Note: While the queue itself is thread-safe, it only protects the data while it is inside the queue. The moment a worker thread dequeues an item and uses it to modify a completely separate shared global variable, you lose all protection.

- **Join vs detach:** A joinable thread retains some resources until another thread calls `join()`. A detached thread runs independently, and its return value cannot be collected

```
0 my $result = $thread->join();
1 $thread->detach();
```

Use `join()` when you need the result or must ensure the work completes. Use `detach()` only for genuinely independent background work.

- **Identifying threads:**

```
0 my $id = threads->tid();
1 print "Current thread ID: $id\n";
```

The original thread has ID 0. Worker threads receive other numeric IDs.

You can inspect existing threads with

```
0 my @joinable = threads->list(threads::joinable);
1 my @running  = threads->list(threads::running);
```

A running thread is actively executing code, while a joinable thread is an active or finished thread whose resources have not yet been cleaned up by a `join()` or `detach()` call.

- **Zombie threads:** If you do not join or detach a thread, you create what is commonly known as a zombie thread, causing resource leaks and potentially causing your program to crash.

When a thread finishes executing its function, its internal stack memory, register states, and exit status are not automatically freed if it is joinable.

- The operating system keeps these structures alive because it assumes the parent thread will eventually call `join()` to read the exit value.
- If you continuously spawn threads without joining or detaching them, you will rapidly deplete the system's memory allocation and file descriptors.
- **Error handling:** If a thread terminates with die, `join()` reports the failure. It is usually clearer to handle errors inside the thread:

```
0 my $thread = threads->create(sub {
1     my $result = eval {
2         perform_operation();
3     };
4
5     return {
6         result => $result,
7         error  => $@,
8     };
9 });
10
11 my $outcome = $thread->join();
12
13 die $outcome->{error} if $outcome->{error};
```

System interaction

- **system()**: The `system()` function allows an external command to be executed. This command will inherit Perl's standard filehandles (STDIN, STDOUT and STDERR) and have control of the console until it terminates. This means that `system()` can be used to launch interactive commands such as editors if desired. Perl will wait for the command specified to terminate before continuing:

```
0  # Open a file for the user to edit.
1  system('vi somefile.txt'); # Or 'notepad somefile.txt'
2
3  # Read the contents of the file once the user has finished.
4  open(my $input_fh, '<', 'somefile.txt') or die "$!";
```

The `exec()` function works in a similar way, but on success your program is replaced with the command specified. This means that `exec()` never returns on success. The `exec()` function is most useful when writing wrapper scripts that wish to establish a certain state before executing another program.

If the command specified by `system()` could not be run the error message will be available via the special variable `!`. This value is not set if the command can be run but fails during runtime. The return status of the command can be found in the special variable `?`, which is also the return value of `system()`. This value is a 16-bit status word which needs to be unpacked to be useful, as demonstrated in the example below.

```
0  system("/path/to/some/command");
1  if ($?) { # A non-zero exit code means failure
2      # -1 means the program didn't even start!
3
4      if($? == -1) {
5          print "Failed to run program: $!\n";
6      }
7
8      else {
9          print "The exit value was: " . ($? >> 8) . "\n";
10         print "The signal number that terminated the program
11             ↪ was: "
12             . ($? & 127) . "\n";
13         print "The program dumped core.\n" if $? & 128;
14     }
15 }
```

- **Security concerns:** Just like open the traditional form of calling `system` and `exec` have security issues due to shell expansion. For example consider the following code:

```

0  print "Please give me a file you want to see: ";
1
2  my $filename = <>; # lets pretend: $filename="fred; rm -rf
   ↪ /home/pjf;"
3
4  chomp($filename);
5  system("cat $filename");

```

In this case, due to shell expansion, the shell will receive the commands:

```

0  cat fred
1  rm -rf /home/pjf

```

and if our program had sufficient permissions to delete pjf's home directory, it would.

As a result, there is another, safer, form of system and exec that bypasses the shell. If you give system or exec a list it assumes that the first element is the command to execute, and every other element is an argument to that command. These arguments are not passed to the shell, and so shell expansion will not occur. So:

```

0  system('cat', '*.txt');

```

will give the "*.txt" filename to cat rather than all files with a .txt extension. This is essential in cases like the above where the command may be passed in from a user. In this case, if the file "*.txt" does not exist then we'll receive a non-zero return code. Exec fails and returns false only if the command (in this case cat) does not exist. If the file does not exist, the user will receive cat's error message

It's always possible to use Perl's glob function to expand filenames for us, without shell intervention:

```

0  system("cat", "*.txt");

```

- **IPC::System::Simple:** The IPC::System::Simple module (available from the CPAN) takes the hard work out of running shell commands:

```

0  use IPC::System::Simple qw(run);
1
2  run("some_command");

```

The run function will execute the command provided and check the result. If the command fails to start, dies from a signal, dumps core, or returns a non-zero exit status, then IPC::System::Simple will throw an exception. Unless you take steps to prevent it, a failure from this command will cause your program to die with an error. If you want to capture the error, you can do so:

```

0  # The 'eval' block allows us to capture errors, which
1  # are then placed in $?. If any of the commands below
2  # fail, the 'eval' is exited immediately. This means if
3  # we fail to backup the files, we won't delete them.
4
5  eval {
6      run('backup_files');
7      run('delete_files');
8  };
9
10 if ($?) {
11     warn "Error in running commands: $?\n";
12 }

```

You can also use `IPC::System::Simple` to execute commands that can return a range of acceptable exit values:

```

0  use IPC::System::Simple qw(run);
1
2  # Run a command, insisting it return 0, 1 or 2:
3  run( [0,1,2], "some_command");
4
5  # Run a command and capture its exit value:
6  my $exit_value = run( [0,1,2], "some_command");
7
8  # Specify return values using '..' notation:
9  my $exit_value = run( [0..2], "some_command");

```

Just like regular system, the `run` command uses the standard shell when running a single command, or invokes the command directly when called in a multiple argument fashion:

- **Backticks:** A new set of quotes, called backticks, can be used to interpolate variables then run the resultant string as a shell command. The output of that command can then be printed, assigned, and so forth.

Just as the `q()` and `qq()` functions can be used to emulate single and double quotes and save you from having to escape quotes that appear within a string, the equivalent function `qx()` can be used to emulate backticks.

Backticks are different to the `system()` command, in that they capture the output of the command they execute, as opposed to passing it through to the user.

When called in a scalar context backticks return the output of the command they execute as a string with possibly embedded newlines. When called in a list context, the output is returned as a list with each separate line of output being a new list element.

The return status of commands called by using backticks can be determined by examining `$?` in the same way as the `system()` example above.

- **Tainted input:** Perl's `-T` flag can be used to check for unsafe user inputs.

```
0  #!/usr/bin/perl -Tw
```

-T stands for "taint checking". Data input by the user is considered "tainted" and until it has been modified by the script, may not be used to perform shell commands or system interactions of any kind. This includes system interactions such as `open()`, `chmod()`, and any other built-in Perl function which interacts with the operating system.

The only thing that will clear tainting is referencing substrings from a regexp match. Here's an example.

```
0  #!/usr/bin/perl -Tw
1  use strict;
2
3  $ENV{PATH} = "/bin:/usr/bin"; # Taint requires we set our
   ↪ path.
4
5  print "Who do you want to finger?\n";
6  my $username = <STDIN>;
7  chomp($username);
8
9  # Check $username to make sure it's clean, then finger.
10 if ( $username =~ /\^(\\w{1,8})$/ ) {
11     # $1 is the contents of the first set of
12     # parentheses in the regexp.
13
14     print qx(finger $1);
15 }
16
17 else {
18     print "That was not a valid username!\n";
19 }
```

You can also untaint data by capturing the match in a list context.

If you've been calling your Perl program from the command line with `perl program.pl` you'll be told that you're turning taint checking on too late, even if you've put it in your shebang line.

This is because Perl wants to know that you want to use taint checking as soon as possible. The way to fix this is to include the `-T` option in your call, so:

```
0  perl -T program.pl.
```

Under Perl 5.8.0 and above, there is also the `-t` switch, which causes tainted operations to generate warnings instead of errors. This is no substitute for real taint checking, but can be useful if you're trying to lock down legacy code and see which areas require attention.

- **Safe.pm:** For greater security when using unknown (and possibly hostile) code, or for writing code which adheres to strict standards about what it's allowed to do, there is the Safe module. This module allows the creation of compartments in which Perl code can be evaluated. These compartments allow you to define explicitly what the code run within them may and may not do. For example, you may deny access to the file system so that the code may not read or write to files. Or you may only permit the code to use certain operators such that it may add and subtract but not divide, for example. Attempts by the code to perform forbidden tasks result in a compilation error at compile time and a fatal error at run time.

Command line switches

- **One-liner and input-processing switches**

Switch	Meaning
'-e CODE'	Execute Perl code from the command line
'-E CODE'	Like '-e', but enables optional features for the installed Perl version
'-n'	Read input line by line and execute the program for each line
'-p'	Like '-n', but print each line afterward
'-a'	Autosplit each input line into the '@F' array; normally used with '-n' or '-p'
'-F PATTERN'	Set the autosplit delimiter used by '-a'
'-0[OCTAL]'	Set the input record separator '\$/'
'-g'	Slurp input: read an entire file at once by setting '\$/' to 'undef'
'-l[OCTAL]'	Enable automatic line-ending processing
'-i[EXT]'	Edit files in place; 'EXT' creates backup files
'-s'	Perform primitive command-line switch parsing into variables

- **-s:** perl -s is a command-line flag that enables rudimentary switch parsing for arguments passed to a script

When you run a script with the -s switch, Perl looks at the arguments on the command line that appear after the script name. If it finds any arguments starting with a hyphen (like -xyz or -size=45), it automatically removes them from the global arguments array (@ARGV) and creates matching scalar variables inside your script

If you pass a switch without a value, Perl creates a variable and sets it to 1 (true). If you use an equals sign (=), Perl sets the variable to whatever follows it.

- **-l:** the perl -l command-line flag enables automatic line-ending processing.

It is one of the most popular flags for writing clean command-line one-liners because it handles two tedious tasks automatically: it chops off trailing newlines from inputs and applies them back to outputs.

When used alongside the loop flags (-n or -p), -l automatically calls chomp on every line read from your input files or standard input. This cleanly removes the trailing newline (\n) so you can manipulate the text without worrying about line breaks.

It alters the behavior of the print statement. Normally, Perl's print requires you to manually add \n to the end of a string. With -l, Perl sets the output record separator (\$\n) to match the input record separator (\$/). As a result, every print statement automatically appends a newline.

The optional octal value passed to `perl -l` specifies the Output Record Separator (`$\`) as an octal number.

- **Module and library switches:**

Switch	Meaning
'-I DIRECTORY'	Add a directory to the beginning of '@INC'
'-M MODULE'	Load a module, approximately equivalent to 'use MODULE'
'-m MODULE'	Load a module without importing its symbols
'-M-MODULE'	Disable a module or pragma using 'no MODULE'
'-m-MODULE'	Equivalent to 'no MODULE ()'

- **Checking, warnings, and security:**

Switch	Meaning
'-c'	Check syntax without executing the main program
'-w'	Enable warnings globally
'-W'	Enable all warnings, regardless of local settings
'-X'	Disable all warnings
'-t'	Enable less strict taint checking
'-T'	Enable full taint checking
'-U'	Permit certain unsafe operations
'-f'	Disable execution of the user's site-customization script

- **Debugging switches:**

Switch	Meaning
'-d'	Run the program under Perl's debugger
'-d:MODULE'	Use another debugger or debugging module
'-D FLAGS'	Enable internal Perl debugging flags; generally requires a debugging build
'-u'	Dump core after parsing; largely obsolete and platform-dependent

Idioms

- **Representation of true and false:** Perl does not have a native true or false keyword. Instead, it determines a value's truth by evaluating what it is.

Only a few specific values are considered false in Perl:

- The number 0 (or 0.0)
- The string "0"
- The empty string ""
- undef (undefined)
- An empty list ()

Everything else is true. When you use a single logical NOT (!), Perl evaluates the operand's truth and returns an absolute boolean result.

- If you pass it something false, it returns 1 (Perl's standard "true" value).
- If you pass it something true, it must return a "false" value. By language design, Perl's logical operators return the empty string (") to represent false.

21.1 Secret Operators

- **Bare number flip-flop:** Print lines 5 through 10 of a file:

```
o perl -ne 'print if 5..10'
```

When the flip-flop operator `..` sees a bare integer on either side, it silently compares that integer against `$.`, the current input line number. The expression `5 .. 10` is really `$. == 5 .. $. == 10`.

In list context, `..` is the range operator:

```
o my @nums = (1..5)
```

In scalar context (like inside an if condition), it becomes the **flip-flop operator**. It remembers state between calls. It starts in the OFF state. When the left condition becomes true, it flips ON. When the right condition becomes true, it flops back OFF.

```
o for my $i (0..9) {  
1     $i > 2 .. $i > 8  
2         ? print "${i}\n"  
3         : print "";  
4  
5     ${i}++;  
6 }  
7  
8 =pod  
9 3  
10 4  
11 5  
12 6  
13 7  
14 8  
15 9  
16 =cut
```

When you put a bare number where the flip-flop expects a condition, Perl doesn't treat it as a boolean. It compares it to `$.`

- **Eskimo greeting (Butterfly operator):** Count the lines in a file:

```
o perl -lne '}{ print $.'
```

Those two braces break out of the implicit while loop that `-n` creates, and start a new block that runs exactly once, after all input has been processed. An END block without typing END.

Every line of input becomes `$_`, your code runs, next line. The `-p` flag does the same thing but adds `print` at the end of each iteration.

Simple enough. But what if you want to do something AFTER all lines are processed? You could use an `END` block

```
o perl -ne '$sum += $_; END { print $sum }' numbers.txt
```

Or you could use two braces and save yourself some typing.

```
o perl -ne '$sum += $_; }{ print $sum' numbers.txt
```

Perl wraps it into:

```
0 while (<>)
1 {
2     $sum += $_;
3 }
4 {
5     print $sum
6 }
```

- **Maori Farewell:** The butterfly operator gives you an `END` block in one-liners. But what about `BEGIN`? What if you need code to run BEFORE the loop starts?

```
o -M5;
```

Everything after the semicolon runs before your main code. Normally `-M` loads a module, but with `-M5`, something different happens.

`-M5` means

```
o use 5;          # Require Perl version 5 (always true)
```

The semicolon separates the version requirement from arbitrary code. Since we're all running Perl 5+, the version check passes silently.

What comes after the semicolon runs at compile time, before the `-n` or `-p` loop even starts. It's not quite a real `BEGIN` block, but it acts like one for practical purposes.

```
o perl -M'5;print scalar localtime, "\n"' -ne 'print if
  ↪ /error/i' log.txt
```

- **Triple dot flip flop:** The flip-flop in its natural habitat:

```

0  while (<DATA>)
1  {
2      print if m~START~ .. m~END~;
3  }
4
5  __DATA__
6  noise
7  START
8  grab this
9  and this
10 END
11 more noise
12
13 =pod
14     START
15     grab this
16     and this
17     END
18 =cut

```

The .. flips ON when it sees START. It stays on, returning true for every line. When it sees END, it returns true one last time and flips OFF. Simple. For 95% of use cases, that's all you need to know. But the remaining 5% is where things get interesting.

.. checks the right-hand condition on the same line the left-hand condition triggers.
 ... waits until the next line to start checking the right-hand condition.

- .. Can flip ON and OFF on the same line
- ... Guarantees at least two evaluations (the trigger line plus the next)

This only matters when both conditions can match the same line

- **Yada Yada:** The yada yada operator. Perl's way of saying "I haven't written this part yet."

```

0  sub fn {
1      ...
2  }

```

Compiles and runs, but fails if you try to call it.

- **Venus operator:** Force numeric context with

```

0  0+

```

The Venus operator. Named because it looks like the astronomical symbol for Venus

```

0  my $str = "42";
1  my $num = 0 + $str;

```

Now \$num is numerically 42, not the string "42". Perl usually handles string/number conversion automatically. But sometimes you need to force it

- **Goatse operator** `=()`: Used to force an expression on its right-hand side into a list context while evaluating the entire assignment in a scalar context. It returns the number of items generated by the right-hand side expression

It is most frequently used to count regex matches in global matching mode (`/g`) without storing all the matched text in memory:

```
0 my $text = "cat dog cat bird cat";
1 my $count = () = $text =~ /cat/g;
2 print $count; # Outputs 3
```

- **Flathead**: Forces scalar context, interpolating a single value.

```
0 ${\( EXPRESSION )}
1
2 ( EXPRESSION )      Evaluate expression in scalar context
3 \ ( ... )           Create a reference to that scalar value
4 ${ ... }            Dereference it for interpolation
```

- **Babycart**: Five characters that let you interpolate ANY expression in a string.

```
0 @ {[ ]}
```

```
0 my $x = 5;
1 my $y = 3;
2
3 print "Sum: @ {[ $x + $y ]}"; # Sum: 8
```

```
0 @ {[ EXPRESSION ]}
1
2 [ EXPRESSION ]      Anonymous array reference containing
   ↳ EXPRESSION
3 @ { ... }           Dereference that array reference
```

The expression is evaluated in list context, wrapped in an anonymous array, then immediately dereferenced for interpolation.

- **Bang bang**: The bang bang operator forces any scalar value into a strict boolean 1 or empty string "" (which Perl treats as false). It is simply two consecutive logical NOT (!) operators.

The first ! evaluates the value and flips its truthiness to a strict boolean (making a true value "" and a false value 1). The second ! flips it back.

Perl doesn't have a dedicated boolean type. If a function returns an object, a string, or a number on success, using `!!` cleans up the data so it only returns a clean 1 (true) or "" (false).

```
0 my $user_input = "Hello";
1
2 my $pure_boolean = !!$user_input;
3 print $pure_boolean; # Outputs 1
```

- **Enterprise operator:** Conditional list elements. You want an item in a list only if some condition is true:

```
0 ()x!!
```

```
0 my @args = (
1     '--verbose',
2     ('--debug') x !!$debug,
3     '--output', $file,
4 );
```

- **Double diamond:** The diamond operator has a problem. A security problem.

```
0 perl script.pl 'ls |'
```

That doesn't read a file called "ls |". It executes `ls` and reads its output. Oops.

The regular diamond uses two-argument `open` internally:

```
0 while (<>) # open(ARGV, $ARGV[0])
```

The triple diamond uses three-argument `open`

```
0 while (<<>>) # open(ARGV, "<", $ARGV[0])
```

Standard functions

- **Map:** The Perl map function transforms a list of values into a new list of values by running a block of code or an expression on each element.

You can use map with either a block of code enclosed in curly braces or a simple expression followed by a comma:

```
0 my @new_list = map { BLOCK } @old_list;
1 my @new_list = map EXPR, @old_list;
```

- **Grep:** grep function acts as a list filter. It evaluates a block of code or an expression for every element in a list, setting the default variable `$_` to each element one by one. It then returns only the elements for which the condition evaluates to true.

```
0 my @results = grep { CONDITION } @list;
1 my @results = grep CONDITION, @list;
```

```
0 my @fruits = ('Apple', 'Banana', 'Avocado', 'Cherry');
1
2 # Find fruits starting with 'A'
3 my @a_fruits = grep { /^A/ } @fruits;
```

- **List::Util::reduce:** The reduce function steps through a list two elements at a time, accumulating a single result.

It registers the accumulator as `$a` and the current element as `$b`.

```
0 use List::Util qw(reduce)
1 my $res = reduce { BLOCK } @list;
2
3 my $sum = reduce { $a + $b } @numbers;
4 my $max = reduce { $a > $b ? $a : $b } @numbers;
```

- **Quotemeta:** In Perl, the quotemeta function returns a string with all non-word characters escaped with a backslash. It explicitly targets any ASCII character not matching the `/[A-Za-z0-9]/regularexpression`.

In double-quoted strings or directly inside regular expressions, quotemeta is internally invoked using the `\Q` escape sequence. The escaping continues until it hits a `\E` sequence or reaches the end of the string

```

0 my $literal = "hello.world*";
1
2 # Approach 1: Using the function
3 my $pattern1 = quotemeta($literal); # Returns
   ↪ "hello\\.world\\*"
4
5 # Approach 2: Using \\Q and \\E in a regex match
6 if ($text =~ /\Q$literal\E/) {
7     print "Matched literally!";
8 }

```

\\;Q and \\;E are used to escape (disable) regex metacharacters, forcing the regex engine to treat everything between them as a literal string.

- \\;Q starts the literal quotation (it stands for "quotemeta").
- \\;E ends the quotation block. If you omit \\;E, the literal matching applies until the very end of the regex pattern

Perl examples

- **@F array:** In Perl, the special array @F holds the split fields of each input line when you use the command-line auto-split mode (-a).

By default, -a splits on whitespace, but passing -F allows you to specify a custom delimiter or a regular expression pattern

- **Perl as AWK**

```
o perl -ane 'print "$F[0]\n"
```